

Développement Agile

P. Rachida AIT ABDELOUAHID

An aerial night view of a city skyline, likely New York City, with numerous skyscrapers illuminated. Overlaid on the image are glowing blue and white arcs and dots, suggesting a global network or data flow. A semi-transparent white rectangle is positioned in the center-right, containing the title text.

Contexte général

Qu'est-ce qu'un système ?

- Un système est un ensemble d'éléments interagissant entre eux suivant un certains nombres de principes et de règles dans le but de réaliser un objectif.
- La frontière d'un système est le critère d'appartenance au système.
- L'environnement est la partie du monde extérieur au système.
- Un système est souvent hiérarchisé à l'aide de sous-systèmes.
- Un système complexe se caractérise par :
 - Sa **dimension**, qui nécessite la collaboration de plusieurs personnes ;
 - Son **évolutivité**.

Qu'est-ce qu'un logiciel ?

Un logiciel est:

- un ensemble d'entités (des programmes, des documentations d'utilisation, des informations de configuration, etc.) nécessaires au fonctionnement d'un processus de traitement automatique de l'information.
- un ensemble d'informations relatives à des traitements effectués automatiquement par un appareil informatique. Y sont incluses les instructions de traitement, regroupées sous forme de programmes, des données.
- un « transformateur » d'information. Cette information peut être aussi simple qu'un bit de données ou aussi complexe qu'une présentation multimédia.

Qu'est-ce qu'un logiciel ?

- Un logiciel est en général un sous-système d'un système englobant.
- Il peut interagir avec des clients, qui peuvent être :
 - des opérateurs humains (utilisateurs, administrateurs, etc.) ;
 - d'autres logiciels ;
 - des contrôleurs matériels.
- Il réalise une spécification : son comportement vérifie un ensemble de critères qui régissent ses interactions avec son environnement.

Qu'est-ce que le génie logiciel ?

Le génie logiciel vise à garantir que :

- la spécification répond aux besoins réels de ses clients ;
- le logiciel respecte sa spécification ;
- les coûts alloués pour sa réalisation sont respectés ;
- les délais de réalisation sont respectés.

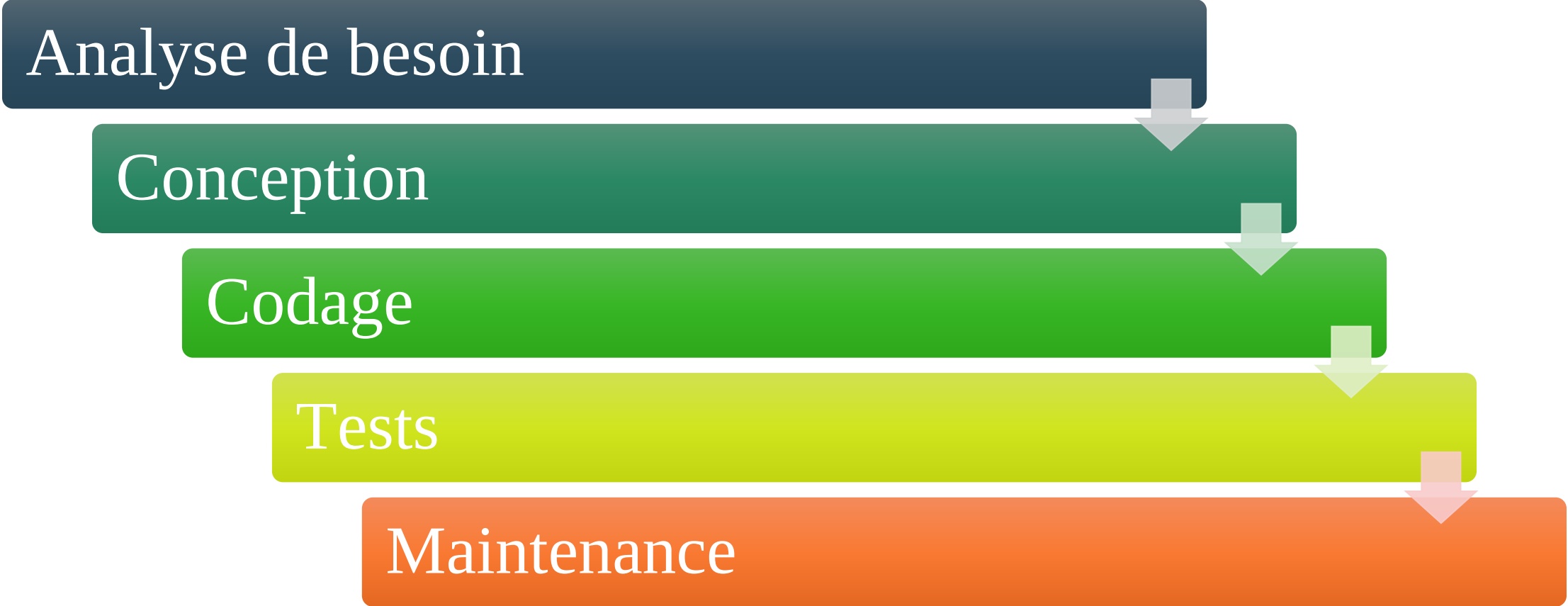
Qu'est-ce qu'un processus ?

Un processus de développement logiciel est un ensemble (**structuré**) d'activités que conduisent à la production d'un logiciel

- Il n'existe pas de processus idéal.
- La plupart des entreprises adapte les processus existants à leurs besoins.
- Ces besoins varient en fonction du domaine, des contraintes de qualité, des personnes impliquées.

Processus de développement logiciel

Analyse de besoin



```
graph TD; A[Analyse de besoin] --> B[Conception]; B --> C[Codage]; C --> D[Tests]; D --> E[Maintenance];
```

Conception

Codage

Tests

Maintenance

Analyse de besoin

- Conditions qui doivent être respectées par un nouveau produit ou un produit altéré
- Aussi appelé spécifications
- Dans les grands projets, c'est composé d'un cahier de charges
- Cette phase est difficile car le client et les développeurs ne parlent pas le même langage
- Le livrable de cette phase est un document de spécification

Conception

- La conception utilise les spécifications pour décider les solutions relatives aux différents problèmes de développement
- La conception décide aussi d'un planning de la solution
- La conception décide de l'architecture de la solution
- Si le produit est centré sur l'utilisateur, la conception propose une ébauche de l'interface utilisateur

Codage

- Le codage est une activité très importante du GL
- Le codage transforme les solutions proposées lors de la conception en un code opérationnel
- La conception et le codage peuvent tous les deux produire du code source, mais c'est l'étape de codage qui rend ce code opérationnel.
- Les techniques de codage dépendent du langage de programmation utilisé et du paradigme.

Tests

- Les tests déterminent la qualité du logiciel
- Les tests déterminent si le logiciel fait ce qu'on attend de lui par rapport aux spécifications
- Plusieurs types de test dont deux principaux : tests unitaires et tests d'acceptation
- Les tests unitaires sont orientés code. Ils se rédigent durant l'activité de codage et se révèrifiant pendant la phase de test
- Les tests d'acceptation vérifient les attentes d'un produit logiciel

Maintenance

- La maintenance consiste à modifier le produit (le logiciel) après sa livraison au client
- Son but est correctif ou évolutif
- La maintenance a deux facettes : organisationnelle et technique
- L'aspect organisationnel concerne l'organisation des équipes pour une réactivité face aux changements
- L'aspect technique concerne une maintenance sans impact négatif sur le produit
- Le degré de maintenance dépend de la qualité de la conception



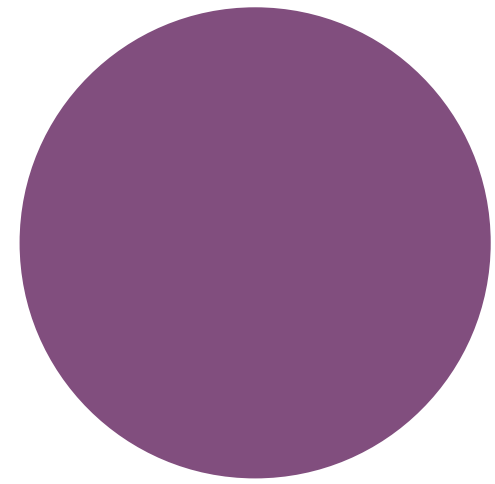
Cycle de vie d'un logiciel (procédé logiciel)

Définition

- Un procédé logiciel (software process) est un ensemble d'activités conduisant à la production d'un logiciel
- Un procédé définit les étapes qui le composent ainsi que leur enchaînement
- Les Cycles de vie de logiciels sont complexes et dépendent fortement des acteurs qui dirigent les activités
- Les activités des procédés ne peuvent être automatisées mais il y a des outils de support, appelés outils CASE (Computer-Aided Software Engineering)

Modèle de procédé

- Un modèle de procédé est une abstraction d'un procédé
- Un modèle décrit le procédé selon une certaine perspective
- Un procédé logiciel est une application d'un modèle pour un projet spécifique, qui peut inclure une certaine adaptation
- Il existe deux types de modèles de procédés:
 - Méthodes classique
 - Méthode agile



Cycle de vie classique

Modèles génériques

Modèles linéaires

modèle en
cascade

modèle en V

Modèles itératifs

modèle de
développement
incrémental

modèle de
développement
en spirale

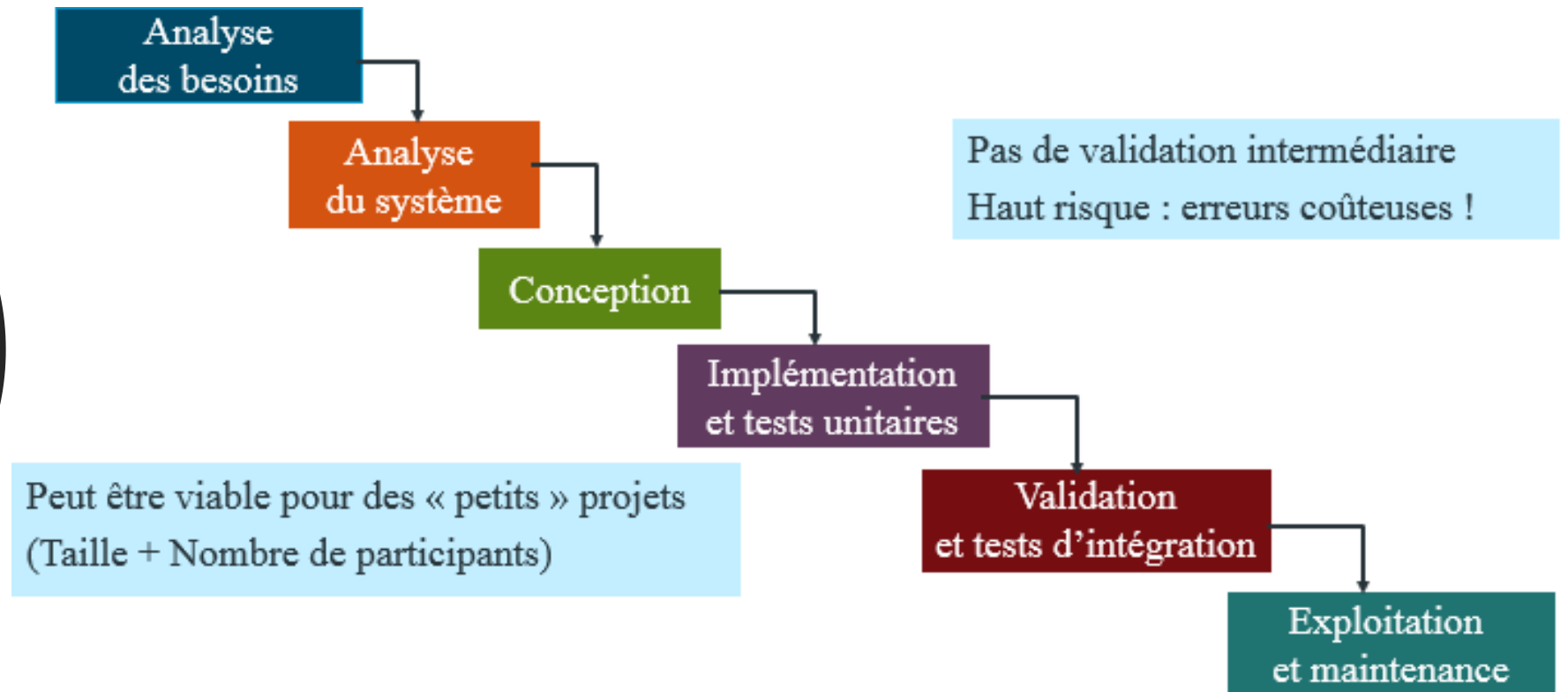
Autres modèles

modèle par
prototypage

prototypage
jetable

prototypage
évolutif

Modèle linéaire : en cascade



Modèles linéaires : en cascade

- Chaque phase se termine à une date précise par la production de certains documents ou logiciels.
- Les résultats sont définis sur la base des interactions entre étapes et activités, ils sont soumis à une revue approfondie

(on ne passe à la phase suivante que s'ils sont jugés satisfaisants)

Modèles linéaires : en cascade

- Certaines phases portent le nom d'une activité
 - signifie que l'activité est essentielle pour cette phase, mais n'impose pas qu'elle n'ait lieu que dans cette étape.
 - D'autres activités interviennent, par exemple le contrôle technique et la gestion de la configuration, tout au long du processus.
- Le modèle original ne comporte pas de possibilité de retour en arrière.
 - a été rajoutée sur la base qu'une étape ne remet en cause que l'étape précédente, ce qui, dans la pratique, s'avère insuffisant.

Modèle linéaire : en cascade Avantages

Facile à utiliser
et à
comprendre

Un procédé
structuré pour
une équipe
inexpérimentée

Idéal pour la
gestion et le
suivi de projets

Fonctionne très
bien quand la
qualité est plus
importante que
les coûts et les
délais

Modèle linéaire : en cascade Inconvénients

Les besoins des clients sont très rarement stables et clairement définis

Sensibilité aux nouveaux besoins : refaire tout le procédé

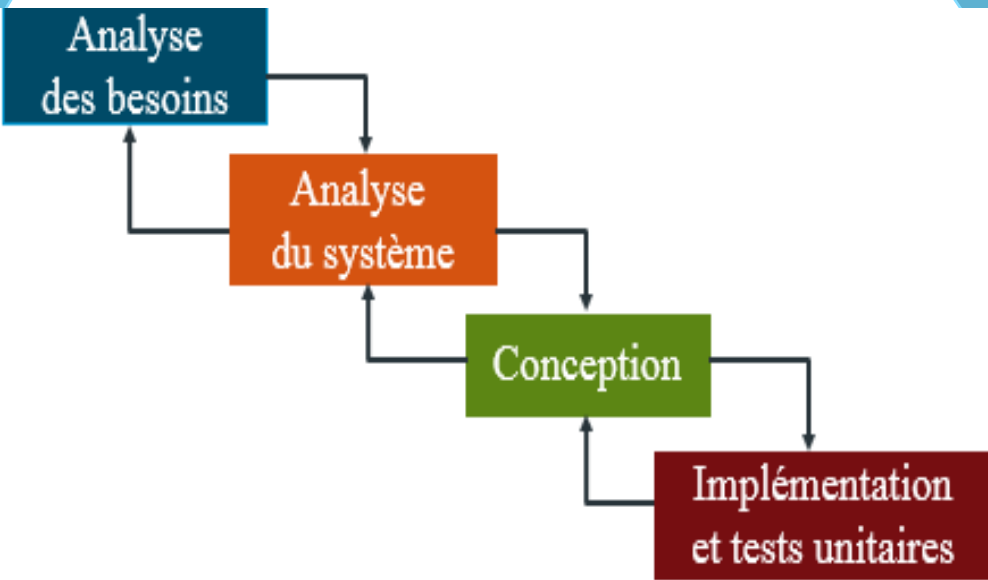
Une phase ne peut démarrer que si l'étape précédente est finie

Le produit n'est visible qu'à la fin

Les risques se décalent vers la fin

Très faible implication du client

Modèles linéaires:Modèle en cascade



Même si on l'étend avec des possibilités de retour en arrière, idéalement limitées à la seule phase qui précède celle remise en cause, le développement reste fondamentalement linéaire.

Modèle linéaire: en cascade Quand l'utiliser ?

Quand les
besoins sont
connus et
stables

Quand la
technologie
à utiliser est
maîtrisée

Lors de la
création d'une
nouvelle
version d'un
produit existant

Lors du
portage d'un
produit sur
une autre
plateforme

Modèles linéaires: en V

Objectifs

Validations intermédiaires pour prévenir les erreurs tardives : meilleure planification et gestion

Principes du cycle de vie en V

- Processus linéaire
- Validation à chaque étape
 - Préparation des protocoles de validation finaux à chaque étape descendante
 - Validation finale montante et confirmation de la validation descendante

Modèles linéaires: en V

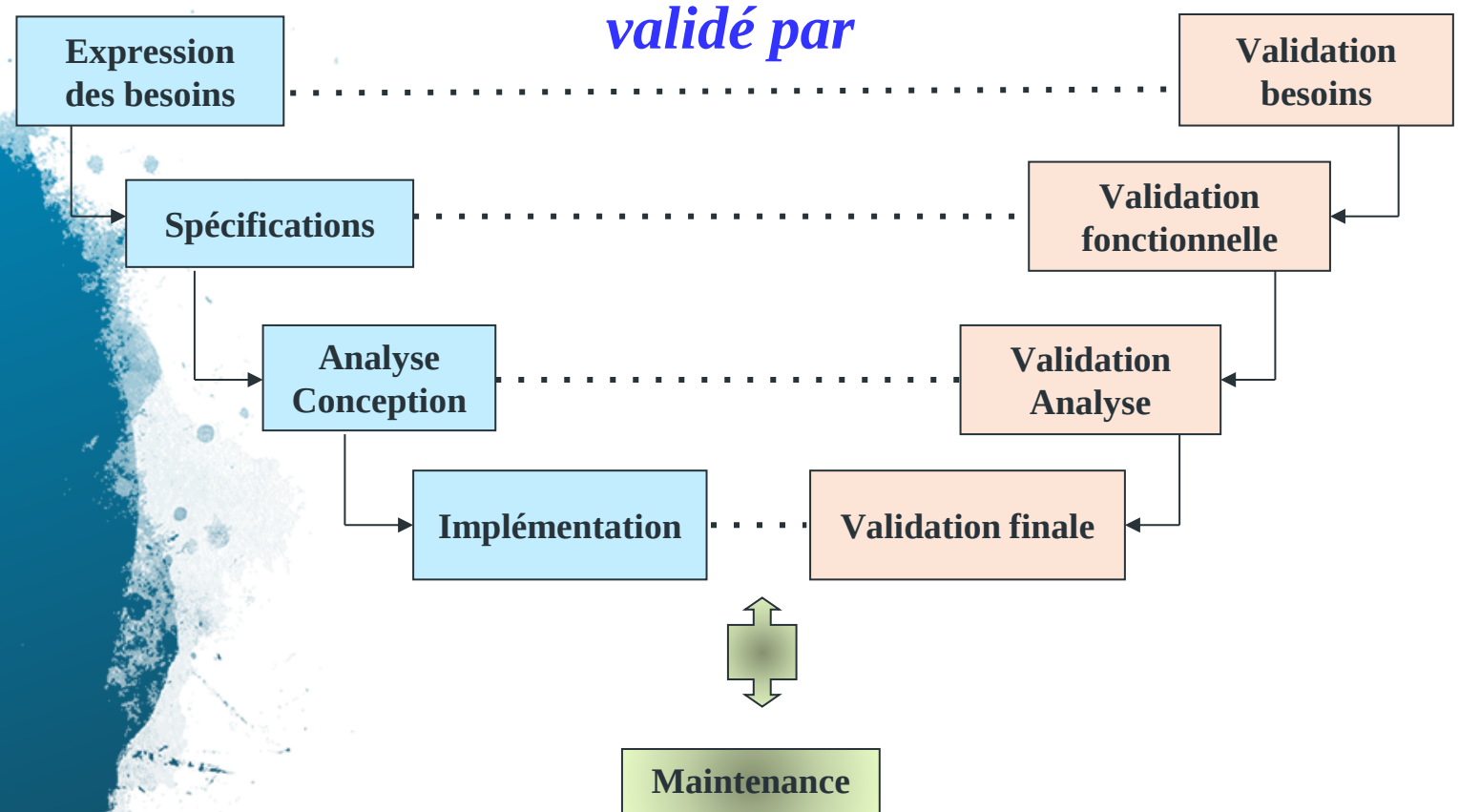
avec toute décomposition doit
être décrite la recombinaison,

la préparation des dernières
phases (validation-vérification)
est explicite par les premières
(construction du logiciel)

toute description d'un composant
est accompagnée de tests qui
permettront de s'assurer qu'il
correspond à sa description.

permet ainsi d'éviter un écueil
bien connu de la spécification :
énoncer une propriété qu'il est
impossible de vérifier
objectivement après la
réalisation.

Modèles linéaires: Modèle en V



Validations des étapes intermédiaires sous forme de documents

Modèle linéaire : en V Avantages

Met l'accent sur
les tests et la
validation et par
conséquent, ça
accroît la qualité
du logiciel

Chaque livrable
doit être testable

Facile à
planifier dans
une gestion de
projets

Facile à utiliser

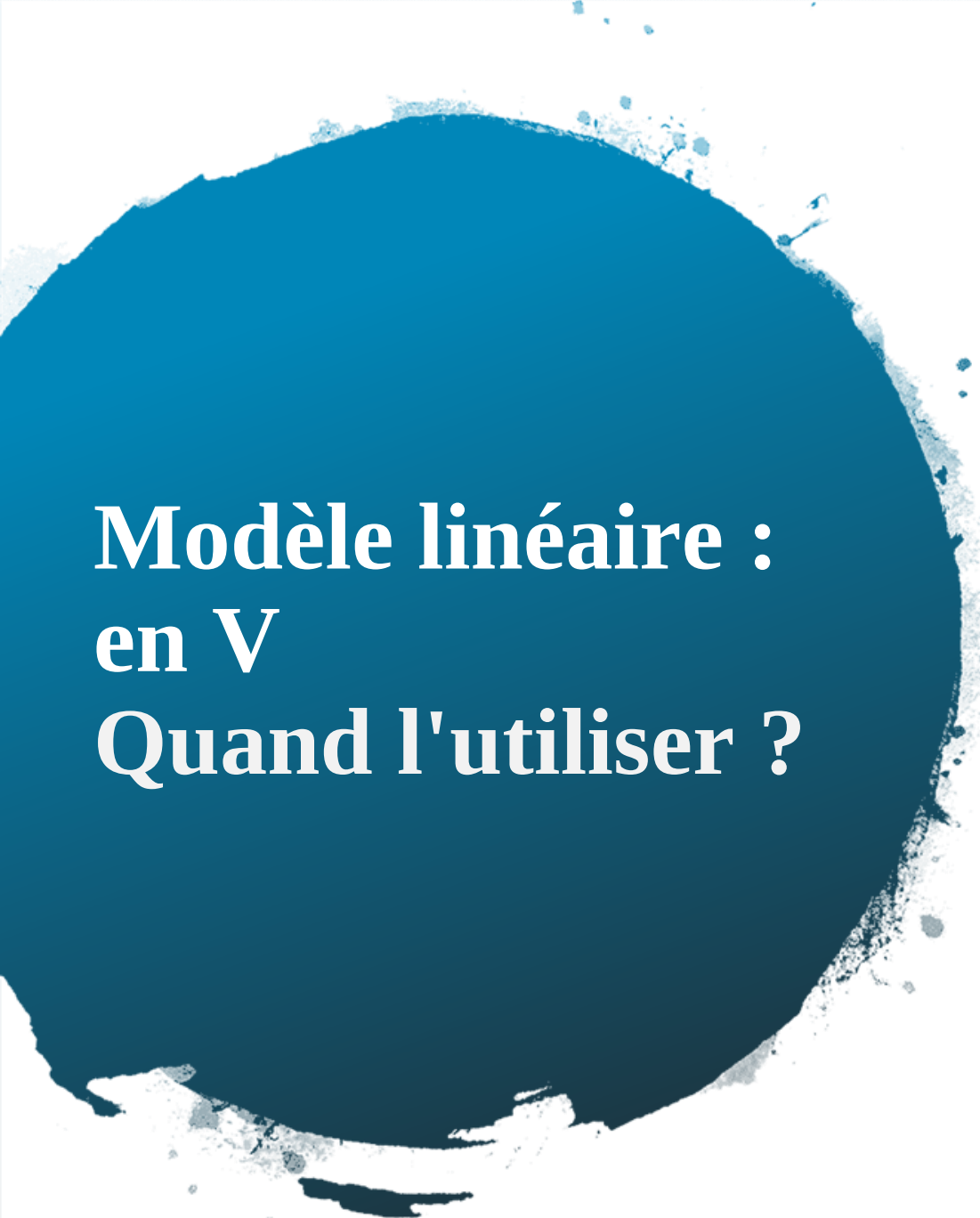


Modèle linéaire : en V Inconvénients

Ne gère pas les activités
parallèles

Ne gère pas explicitement les
changements des
spécifications

Ne contient pas d'activités
d'analyse de risque



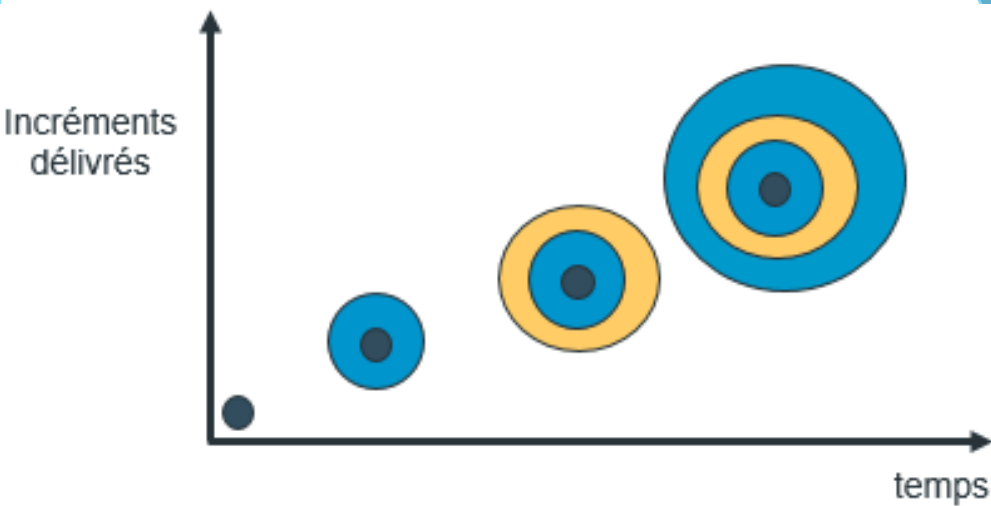
Modèle linéaire : en V Quand l'utiliser ?

Quand le produit à
développer a de très
hautes exigences de
qualité

Quand les besoins
sont connus à
l'avance

Les technologies à
utiliser sont
connues à l'avance

Modèle itératif : incrémentale



Face aux dérives bureaucratiques de certains gros développements, et à l'impossibilité de procéder de manière aussi linéaire, le *modèle incrémental* a été proposé dans les années 80.

Modèle itératif : incrémentale

- Le produit est délivré en plusieurs fois, de manière incrémentale, c'est à dire en le complétant au fur et à mesure et en profitant de l'expérimentation opérationnelle des incréments précédents.
- Chaque incrément peut donner lieu à un cycle de vie classique plus ou moins complet.
- Les premiers incréments peuvent être des *maquettes* (jetables s'il s'agit juste de comprendre les besoins des utilisateurs) ou des *prototypes* (réutilisables pour passer au prochain incrément en les complétant et/ou en optimisant leur implantation).
- Le risque de cette approche est celui de la remise en cause du noyau.

Modèle itératif : incrémentale

- Différents des autres modèles où un logiciel est décomposé en composants développés séparément et intégrés à la fin du processus
- Dans les modèles par incrément un seul ensemble de composants est développé à la fois :
 - des incréments viennent s'intégrer à un *noyau de logiciel* développé au préalable
 - chaque incrément est développé selon l'un des modèles précédents

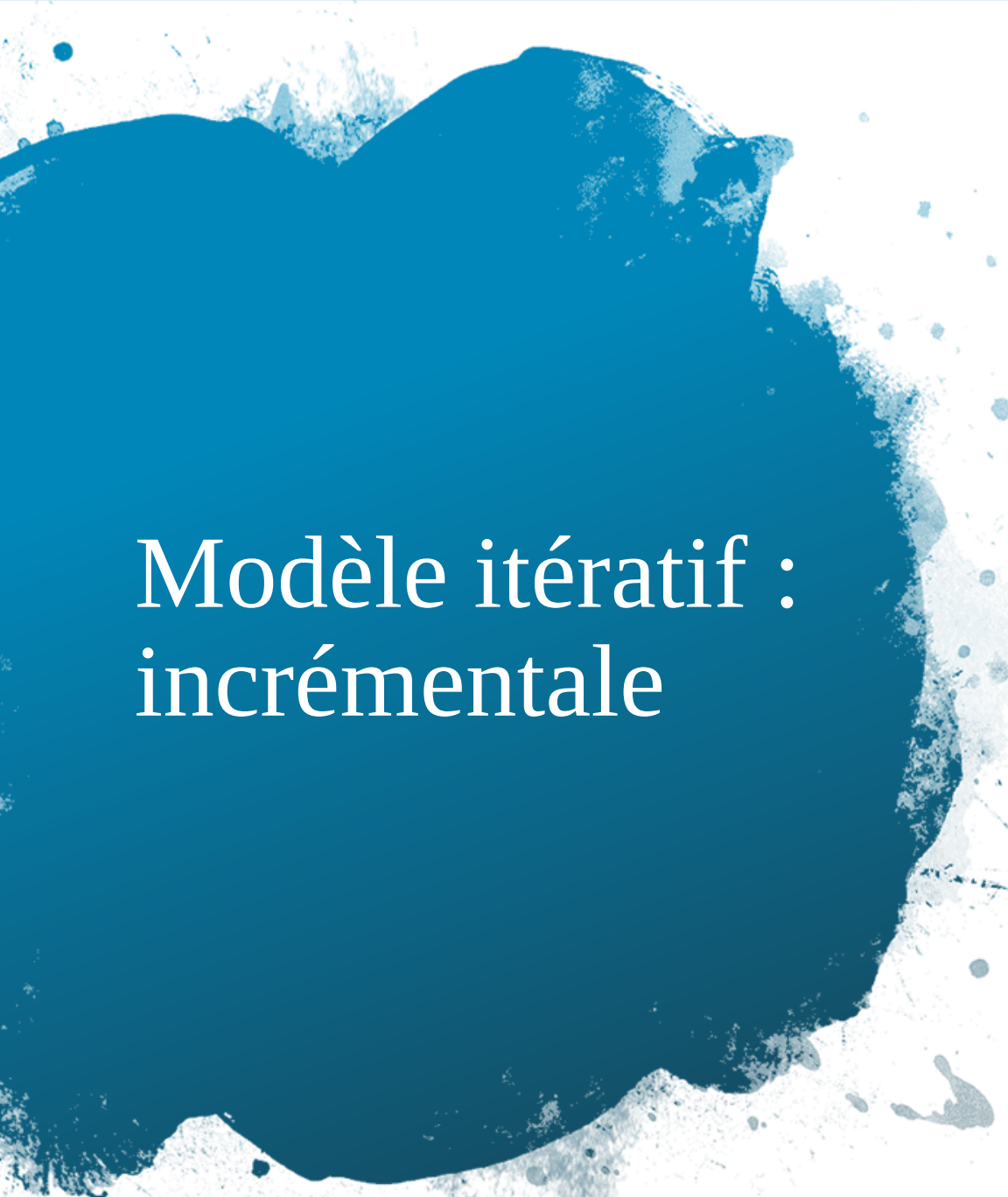
Modèle itératif : incrémentale



Modèle itératif : incrémentale

Avantage s

- Développement de fonctionnalités à risque en premier
- Chaque incrément donne un produit fonctionnel
- Le client intervient à la fin de chaque incrément
- Utiliser l'approche « **diviser pour régner** »
- Le client entre en relation avec le produit très tôt



Modèle itératif : incrémentale

Inconvénients

- Exige une bonne planification et une bonne conception
- Exige une vision sur le produit fini pour pouvoir le diviser en incréments
- Le coût total du système peut être cher

Modèle itératif : incrémentale

Quand l'utiliser ?

- Quand la plupart des spécifications sont connues à l'avances et vont être sujettes à de faibles évolutions
- Quand on veut rapidement un produit fonctionnel
- Pour des projets de longues durées
- Pour des projets impliquant de nouvelles technologies

Modèles itératifs : en spirale

1. Determine
objectives

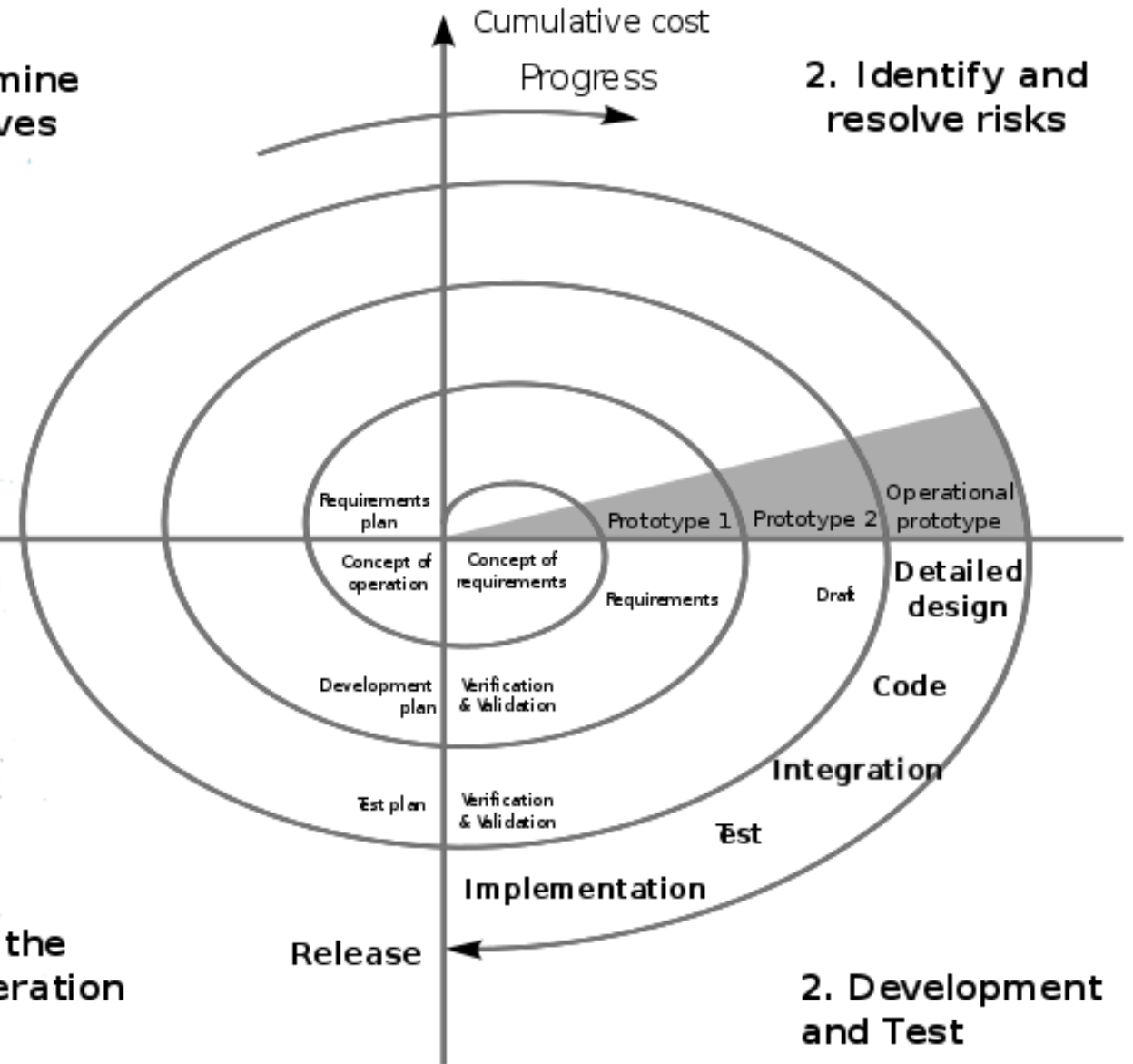
2. Identify and
resolve risks

Review

4. Plan the
next iteration

Cumulative cost

Progress



Modèle itératif : en spirale

Détermination des objectifs

- En terme de fonctionnalité, de performance, de coût, etc.
- Déterminer les alternatives : développer, réutiliser, acheter, sous-traiter, etc.
- Contraintes : coûts, plannings, etc.

Modèle itératif : en spirale

Identification et évaluation de risques

- Etudier les alternatives de développement
- Identification des risques : technologie non maîtrisées, équipe peu expérimentée, planning trop serré, etc.
- Evaluation des risques : voir si les
- risques peuvent impacter le projet et à quel degré

Modèle itératif : en spirale

Développement et test

- Contient pratiquement la plupart des activités : conception, codage, test, etc.

Planification de la prochaine iteration

- Un planning de l'itération
- Un plan de tests

Modèles itératifs : en spirale

Les principaux risques et leurs remèdes, tels que définis par Boëhm

Risque	Remède
Défaillance de personnel	Embauches de haut niveau, formation mutuelle, leaders, adéquation profil/fonction, ...
Calendrier et budgets irréalistes	Estimation détaillée, développement incrémental, réutilisation, élagage des besoins, ...
Développement de fonctions inappropriées	Revue d'utilisateurs, manuel d'utilisation précoce, ...
Développement d'interfaces utilisateurs inappropriées	Maquettage, analyse des tâches, ...
Volatilité des besoins	Développement incrémental de la partie la plus stable d'abord, masquage d'information, ...
Problèmes de performances	Simulations, modélisations, essais et mesures, maquettage, ...
Exigences démesurées par rapport à la technologie	Analyses techniques de faisabilité, maquettage, ...
Tâches ou composants externes défaillants	Audit des sous-traitants, contrats, revues, analyse de compatibilité, essais et mesures, ...

Avantage s

Modèle itératif : en spirale

- Identification rapide des risques
- Impacts minimaux des risques sur le projet
- Fonctions critiques développées en premier
- Feedback rapide du client
- Une évaluation continue du procédé

Modèle itératif : en spirale

Inconvénients

- L'évaluation des risques peut prendre beaucoup de temps
- Le modèle est très complexe
- La spirale peut s'éterniser
- Les développeurs doivent être réaffectés pendant les phases de non-développement
- Les objectifs ne sont pas souvent faciles à formuler

Modèle itératif : prototypage

Idée : fournir rapidement un prototype exécutable pour permettre une validation concrète (ici expérimentale) et non sur papier (document)

- Progressions par incréments successifs de versions successives du prototype : itérations
- Certains prototypes peuvent être « validés » par le client
- Ne dispense pas de fournir des documents intermédiaires

Modèles itératifs: Prototypage jetable

Spécification
schématique



Codage du
prototype



Evaluation du
prototype



Spécification
du système

- Modèle gérable d'un point de vue changement des spécifications
- Les spécifications sont générées par prototypage

Modèles itératifs: Prototypage évolutif

Spécification
schématique



Codage du
prototype



Evaluation du
prototype



Acceptation
du système

Modèle difficile à gérer

Modèle itératif : prototypage

Avantage

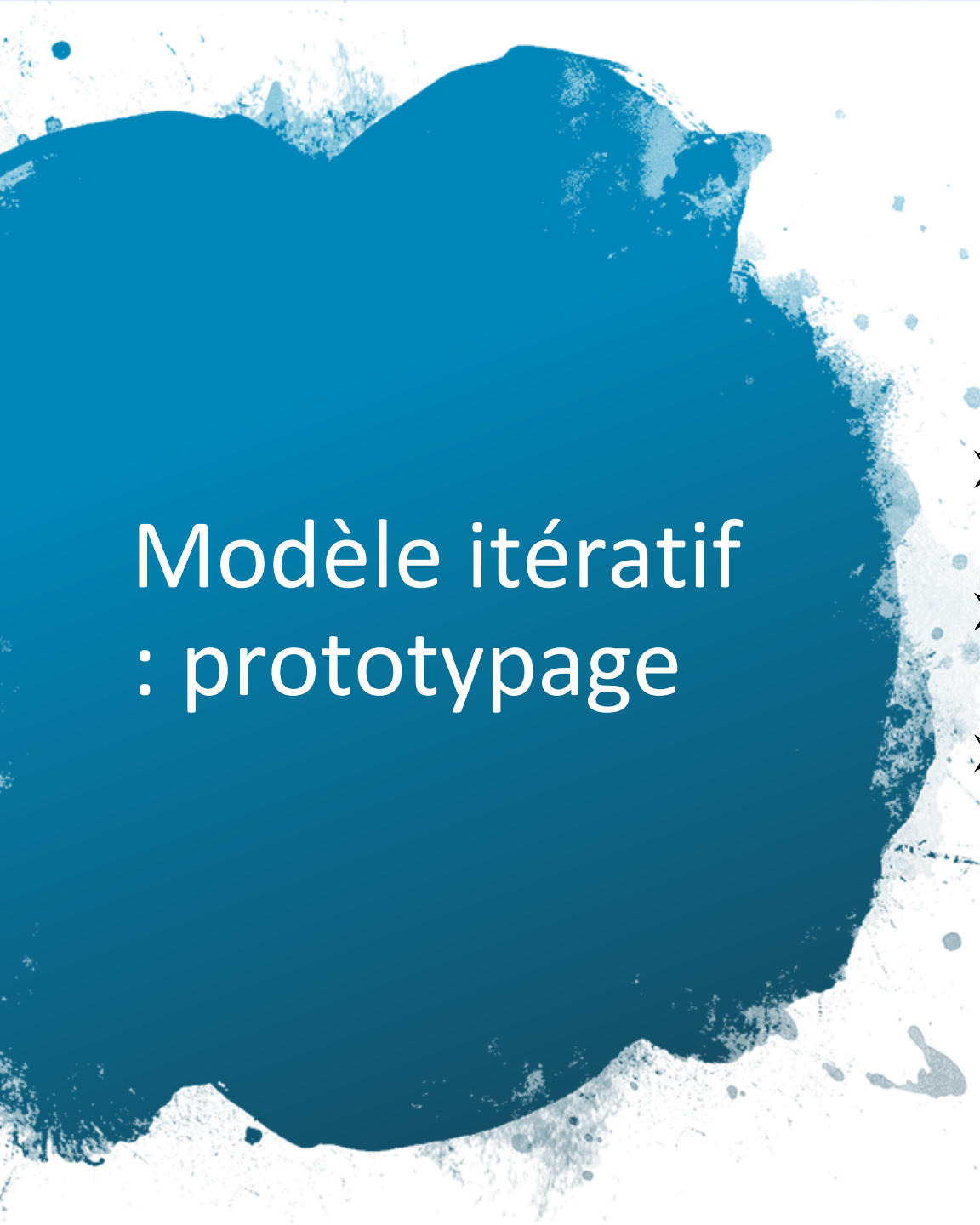
s

- Implication active du client
- Le développeur apprend directement du client
- S'adapte rapidement aux changements des besoins
- Progrès constant et visible
- Une grande interaction avec le produit

Inconvénients

Modèle itératif : prototypage

- Le prototypage implique un code faiblement structuré
- Degré très faible de maintenabilité
- Le processus peut ne jamais s'arrêter
- Très difficile d'établir un planning



Modèle itératif : prototypage

Quand utiliser?

- Quand les besoins sont instables et/ou nécessitent des clarifications
- Peut être utilisé avec le modèle en cascade pour la clarification des besoins
- Quand des livraisons rapides sont exigées

Autres modèles: Transformationnels

Spécification
formelle

Transformation
correcte

Spécification
formelle concrète

Génération de
code

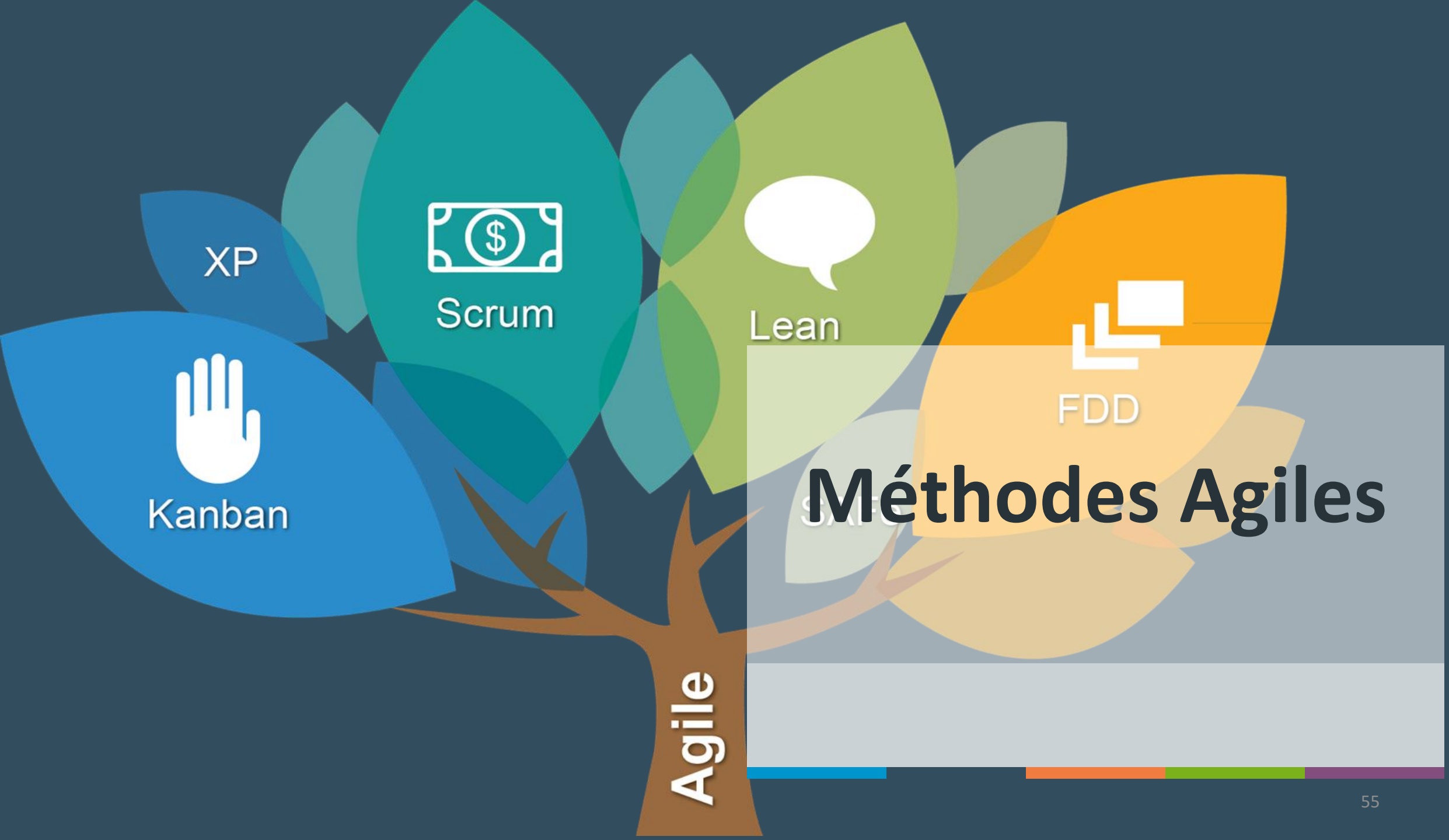
Programme
correcte

- ❑ Modèle gérable
- ❑ Difficile à mettre en œuvre :
Passage du formel au concret =>
nécessite de connaissances avancées
(techniques et formelles)

Autres modèles: Hybrides

Principe :

- Système complexe = composition de sous-systèmes
- Utilisation d'un processus de développement adapté à chaque sous-systèmes
 - Utilisation du modèle à chute d'eau pour les sous-systèmes bien connus
 - Utilisation du modèle de prototypage évolutif pour les sous-systèmes « délicats »



Panorama of agile development methods

XP



FDD



CRYSTAL FAMILY



DSDM



LEAN SD

KANBAN



ASD



ASD

AUP



SCRUM



Méthodes Agiles

- Les méthodes de développement dites « **méthodes agiles** » (en anglais *Agile Modeling*, noté *AG*) visent à réduire le cycle de vie du logiciel (donc accélérer son développement) en développant une version minimale,
- puis en intégrant les fonctionnalités par un processus itératif basé sur une écoute client et des tests tout au long du cycle de développement.
- L'origine des méthodes agiles est liée à l'instabilité de l'environnement technologique
- et au fait que le client est souvent dans l'incapacité de définir ses besoins de manière exhaustive dès le début du projet.

Méthodes Agiles

- Le terme « agile » fait ainsi référence à la capacité d'adaptation aux changements de contexte et aux modifications de spécifications
- intervenant pendant le processus de développement.
- En 2001, 17 personnes posèrent ainsi au point le **manifeste agile** dont la traduction est la suivante

Méthodes Agiles: valeurs

Individus et interactions au lieu de processus et outils

Logiciel fonctionnel au lieu de documentation massive

Collaboration du client au lieu de négociation de contrats

Réagir au changements au lieu de suivre le plan

Méthodes Agiles: valeurs

INDIVIDUS ET INTERACTIONS AU LIEU DE PROCESSUS ET OUTILS

- Les collaborateurs sont la clé du succès
- Les « seniors » échoueront s'ils ne collaborent pas en tant qu'équipe
- Un bon collaborateur n'est pas forcément un bon programmeur. C'est quelqu'un qui travaille bien en équipe
- Une surabondance d'outils est aussi mauvaise que le manque d'outils
- Démarrer petit et investir peu au démarrage
- Construire l'équipe c'est plus important que construire l'environnement

Méthodes Agiles: valeurs

LOGICIEL FONCTIONNEL AU LIEU DE DOCUMENTATION MASSIVE

- Un code sans documentation est un désastre
- Trop de documents est pire que pas de documents
- Difficulté à produire et à synchroniser avec le code
- Souvent les documents sont des « mensonges » formels
- Le code ne ment jamais sur lui-même
- ■ Produire toujours des documents aussi courts que possible

Méthodes Agiles: valeurs

COLLABORATION DU CLIENT AU LIEU DE LA NÉGOCIATION DE CONTRATS

- Très difficile de décrire la totalité du logiciel depuis le début
- Les projets réussis impliquent les clients d'une manière fréquente et régulière
- Le client doit avoir un contact direct avec
- l'équipe de développement

Méthodes Agiles: valeurs

RÉAGIR AUX CHANGEMENTS AU LIEU DE SUIVRE UN PLAN

- Un logiciel ne peut pas être planifié très loin dans le futur
- Tout change : technologie, environnement et surtout les besoins
- Les chefs de projets classiques fonctionnent sur la base de GANTT, PERT et le système de tâches
- Avec le temps, les diagrammes se dégradent car des tâches s'ajoutent et d'autres deviennent non nécessaires
- Une meilleure stratégie est de planifier très court (02 semaines à 01 mois)
- Plannings détaillés pour la semaine à venir, rigoureux pour les trois mois et très vagues au-delà

Méthodes Agiles: principes

Toujours satisfaire le client à travers des livraisons rapides et continues

Bien accueillir tous les changements même les tardifs

Livrer fréquemment un système fonctionnel

Les clients et les développeurs doivent collaborer

Conduire le projet autour d'équipes motivées

La meilleure méthode de faire circuler l'information c'est le contact direct entre collaborateurs

La première mesure d'avancement c'est un logiciel Fonctionnel

Le développement doit être durable et à un rythme constant

La bonne conception et l'excellence technique augmentent l'agilité

Simplifier au maximum

Les meilleurs architectures, besoins et conceptions proviennent d'équipes qui s'organisent d'elles-mêmes

L'équipe s'améliore d'une manière autonome et régulière



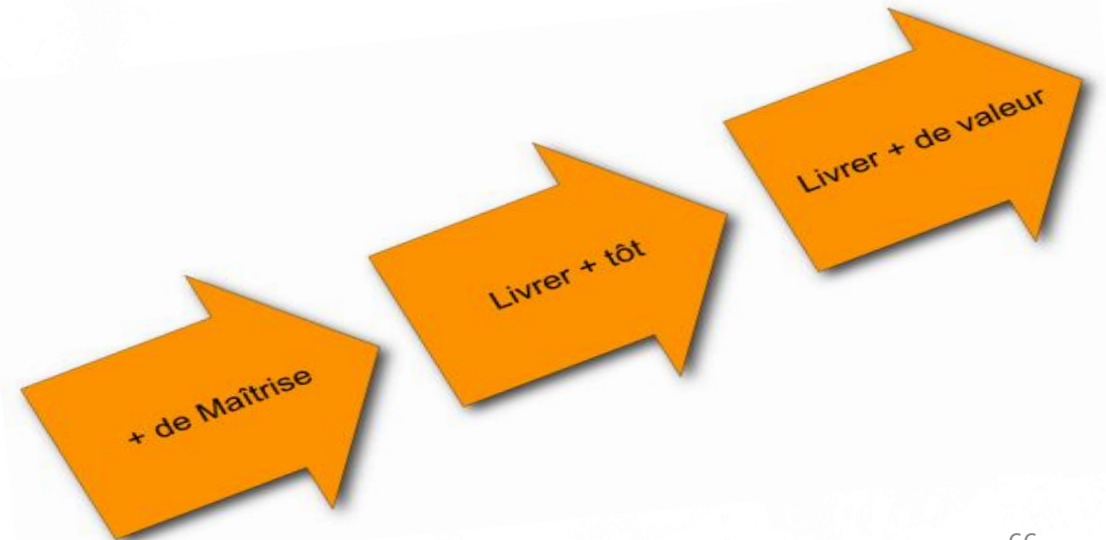
Scrum ?

Scrum

- Méthode itérative et incrémentielle.
- Equipes de 8 personnes.
- Méthode agile la plus utilisée avec eXtreme Programming

Objectifs :

- Satisfaire au mieux les besoins du client
- Maximiser les chances de réussite du projet
- Scrum "is not a process or a methodology, but a path".

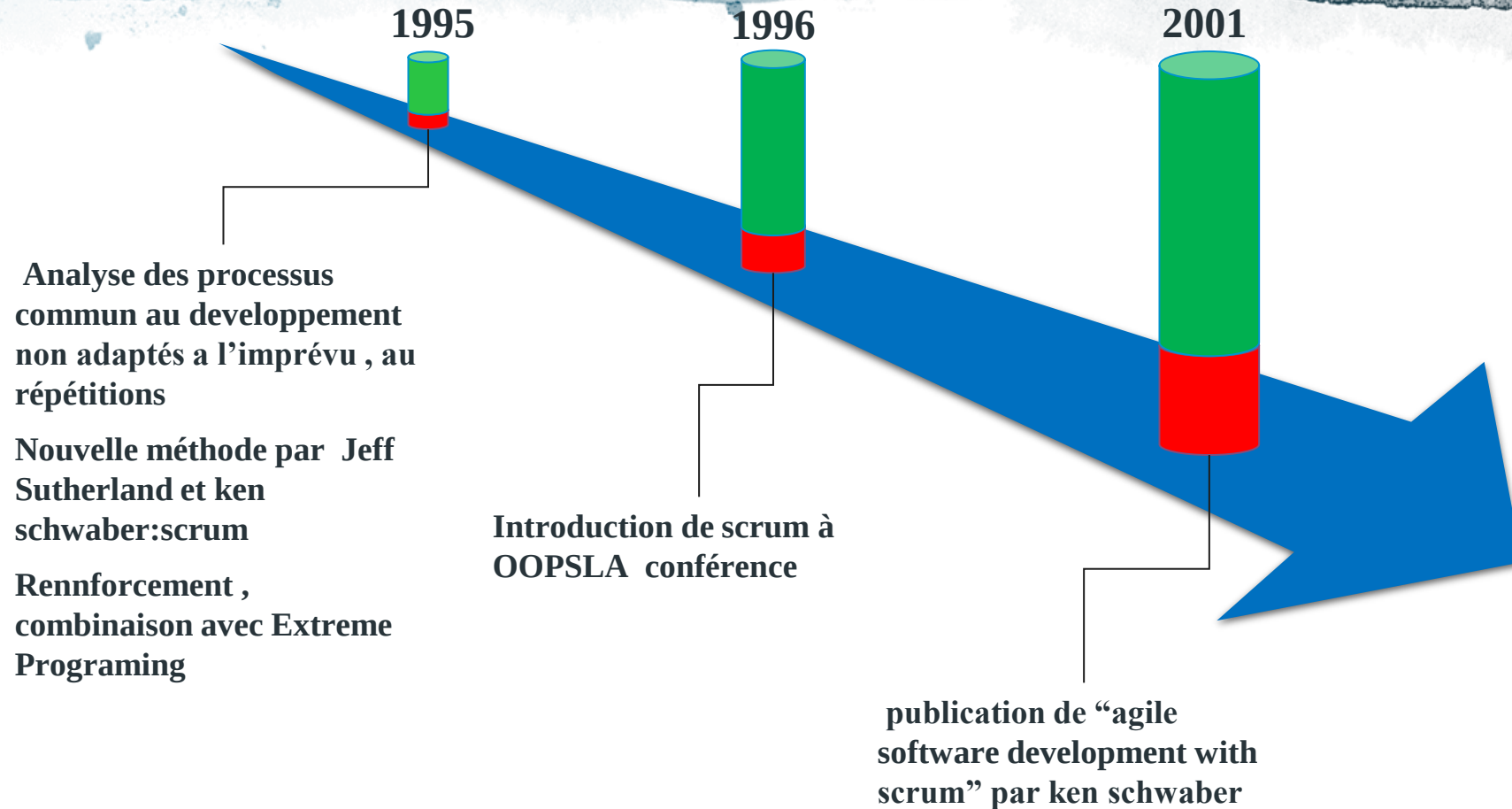


Scrum: Quatre valeurs



- Manifeste pour le développement Agile de logiciels basé sur:
- 1. **Individus et leurs interactions** plus que les processus et les outils
- 2. **Logiciels opérationnels** plus qu'une documentation exhaustive
- 3. **La collaboration avec les clients** plus que la négociation contractuelle
- 4. **L'adaptation au changement** plus que le suivi d'un plan

Aperçu historique



Cycle de vie de la méthode SCRUM

Cycle de développement

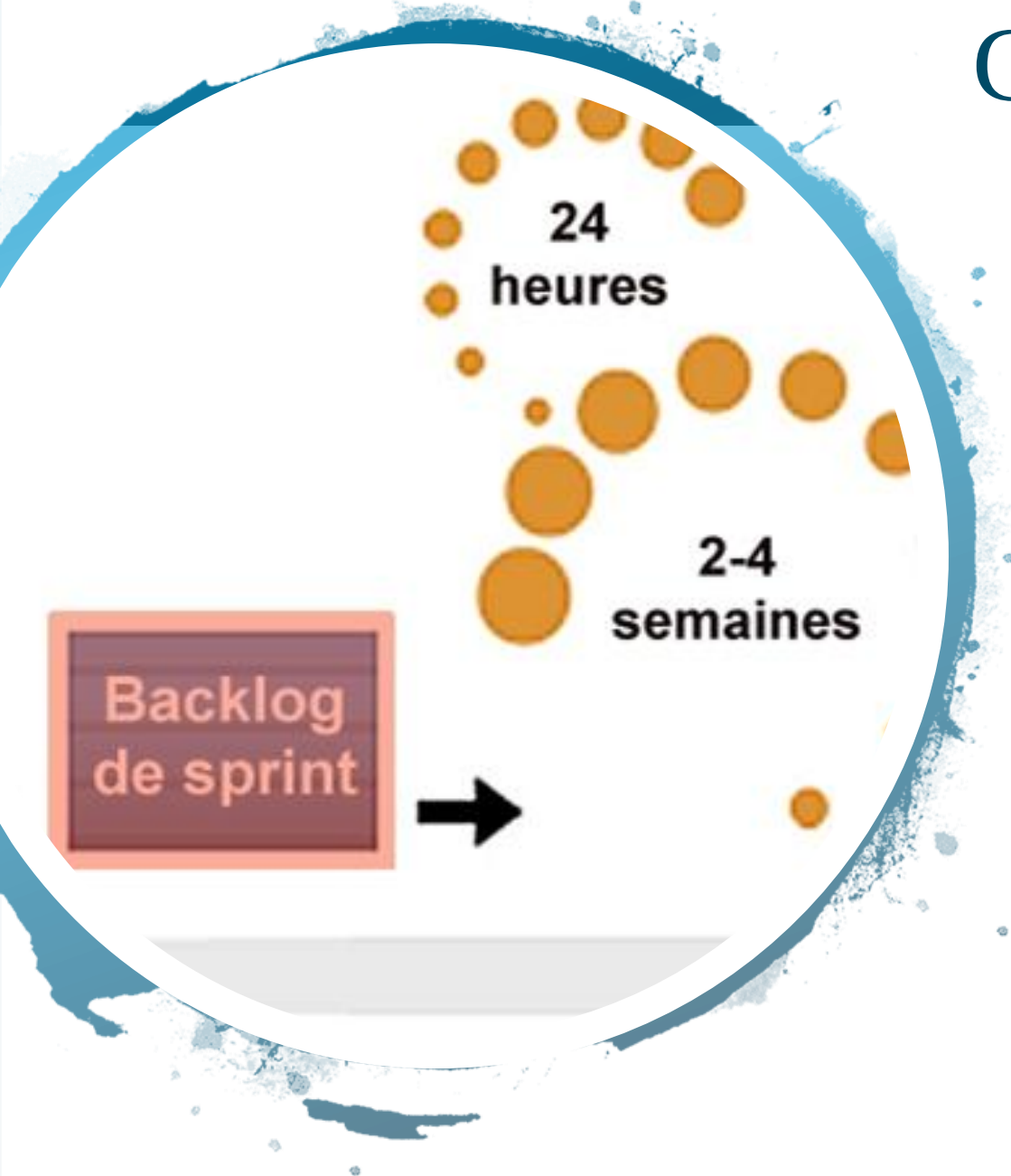


Backlog de produit (catalogue des besoins)

- est une liste priorisée d'exigences, d'histoires, de caractéristiques ou autre. Des produits que le client veut, décrites selon la terminologie du client.

1. ID: un identifiant unique,
2. Nom: une description courte de l'histoire.
3. Importance: attribuée par le directeur de produit à cette histoire.
4. Estimation initiale: l'évaluation initiale de l'équipe sur la quantité de travail.
5. Notes: n'importe quelle autre info,

Cycle de développement



Un backlog de produit
et *un* directeur de
produit doivent exister.

le directeur de produit
devrait comprendre
chaque histoire

Chaque item est alors
divisé en petites tâches
constituant le backlog
du sprint.

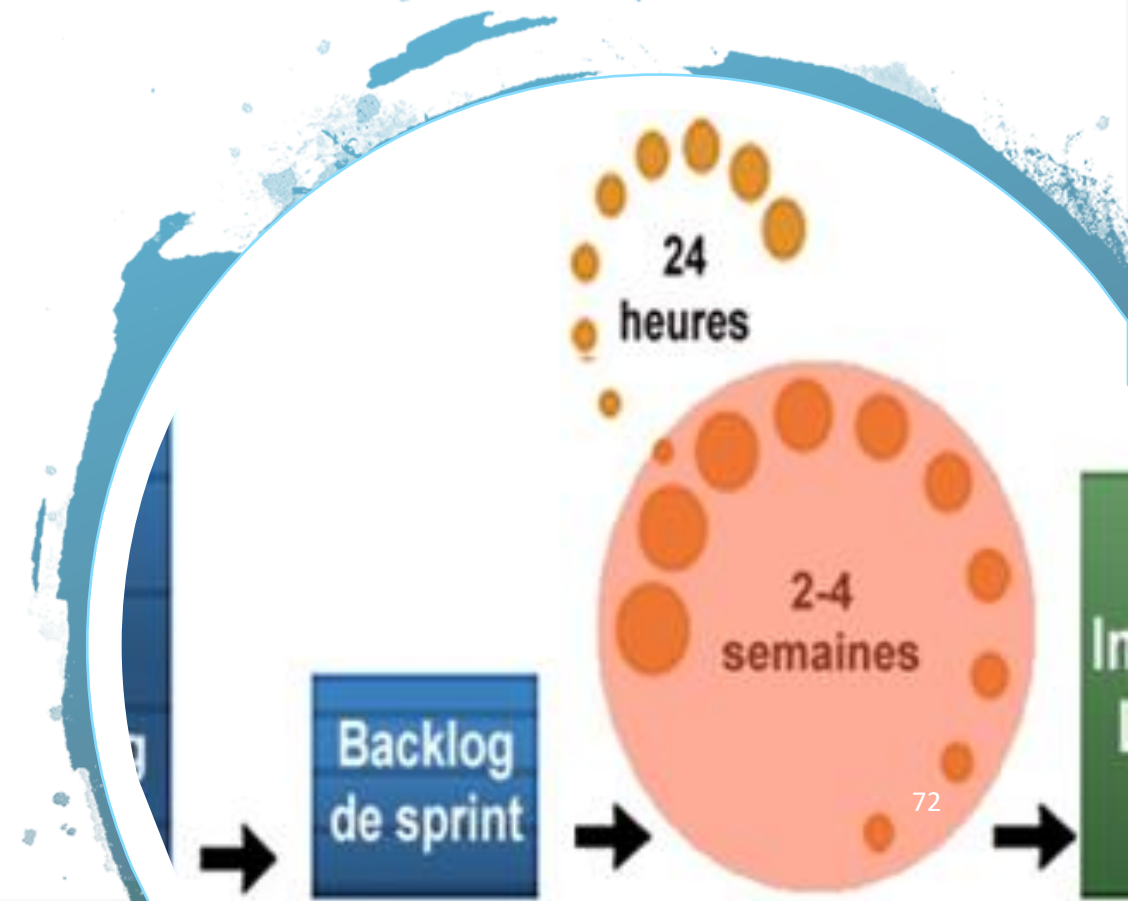
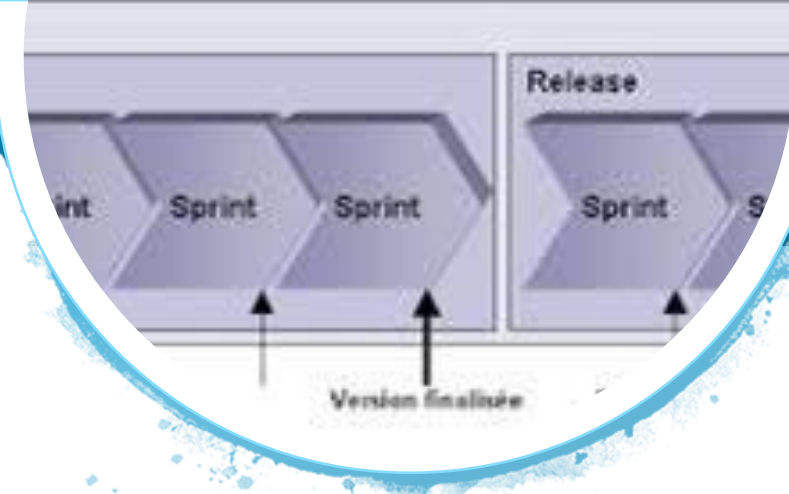
Cycle de développement

1

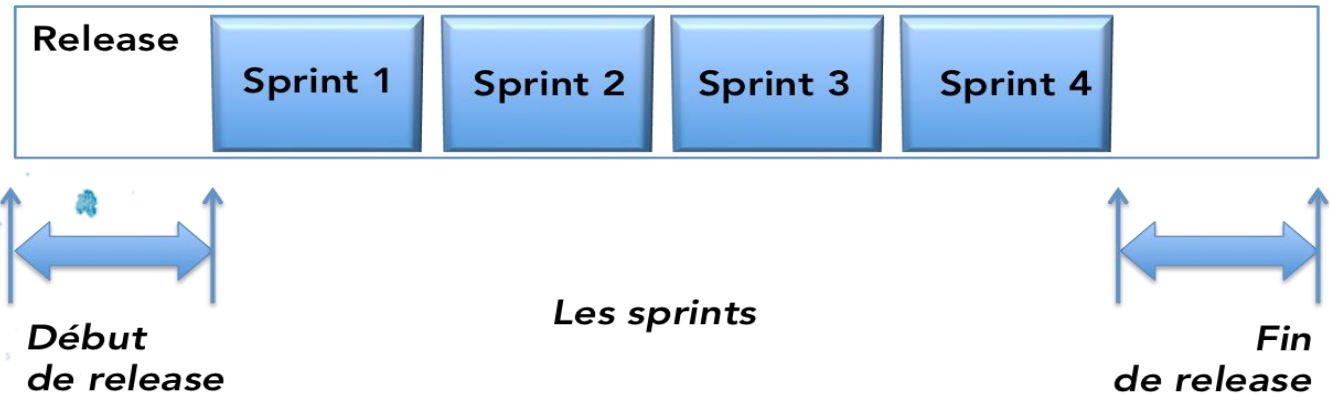
Une itération d'un mois, dont la durée est fixe pendant tout le projet.

2

Chaque sprint permet de développer un sous-ensemble potentiellement livrable du produit final.



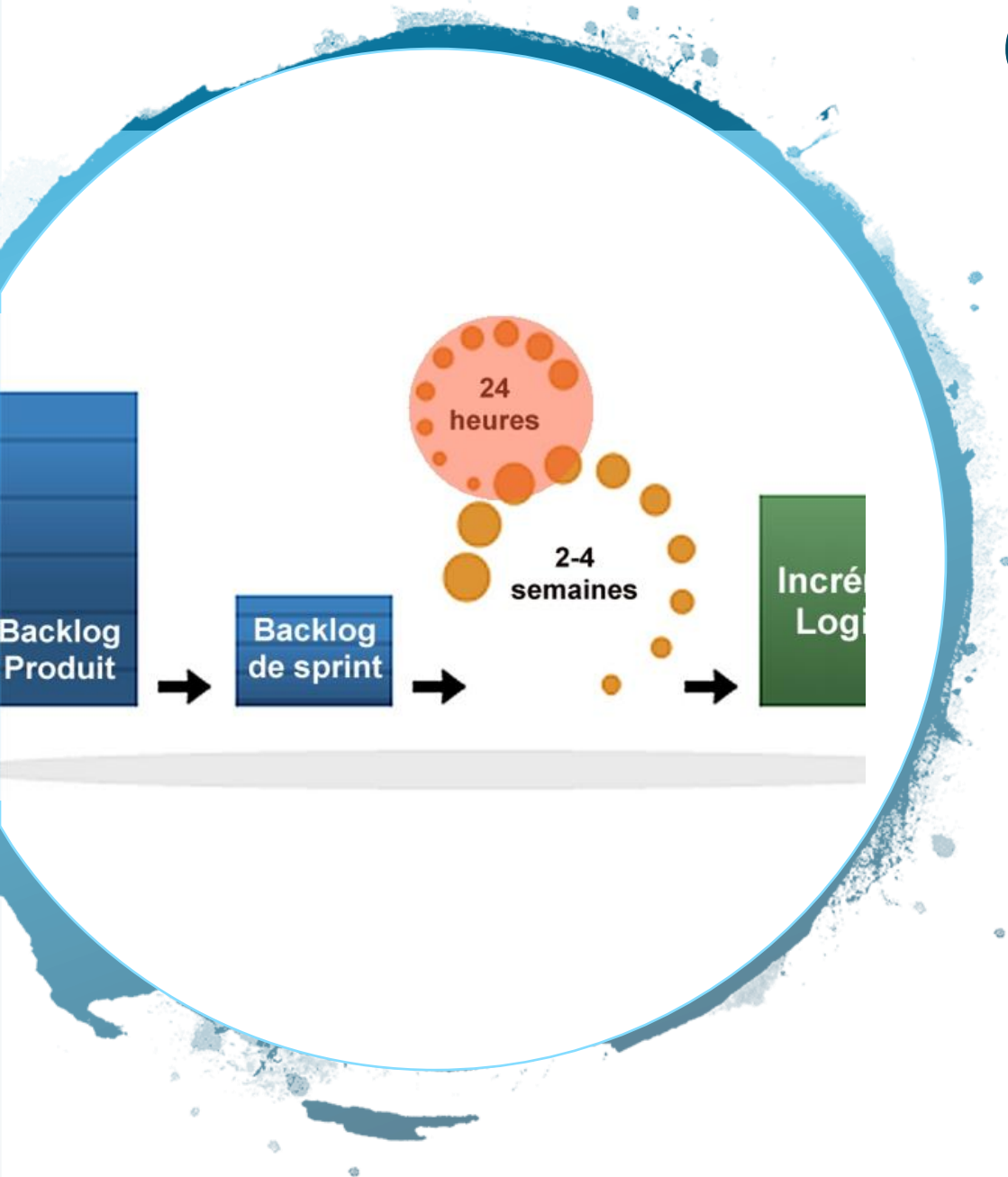
Cycle de développement



Sprint

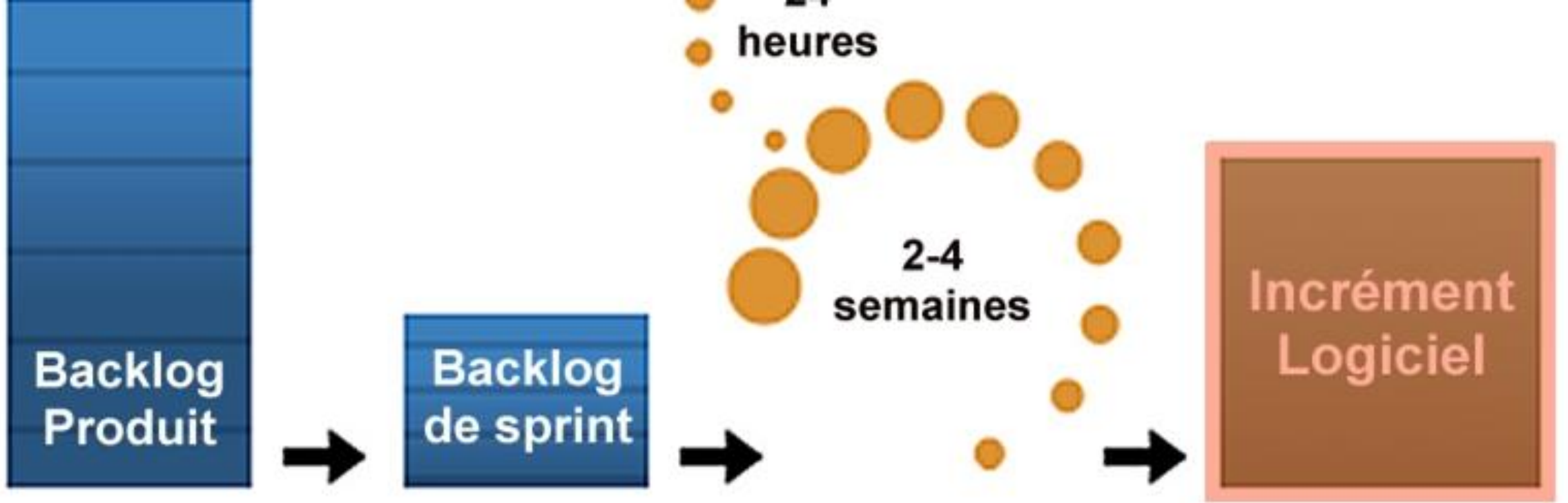
- un bloc de temps fixé aboutissant à créer un incrément du produit potentiellement livrable.
- La durée d'un sprint est de 2 à 4 semaines.
- Ils ont tous la même durée (**timebox**) dans la mesure du possible cela apporte un meilleur rythme.
- Au cours du sprint les changements ne sont pas pris en compte et sont différés jusqu'au prochain sprint
- L'équipe travaille dans toutes les activités de développement pendant un sprint

Cycle de développement



Mêlée quotidienne

- Point de contrôle quotidien de l'équipe
- Interventions régulées – 2 min. par personne



livré au product owner à la fin du sprint.

Cycle de développement

Cycle de développement

Exigences

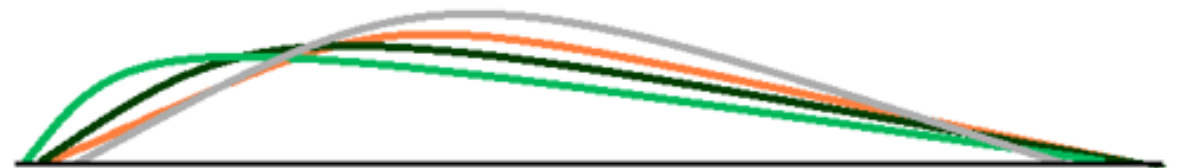
Conception

Code

Test

Les activités se déroulent en parallèle pendant le sprint ...

contrairement au processus séquentiel (activité par phase)



Rôles

Rôles

- Product Owner
- Scrum Master
- Equipe

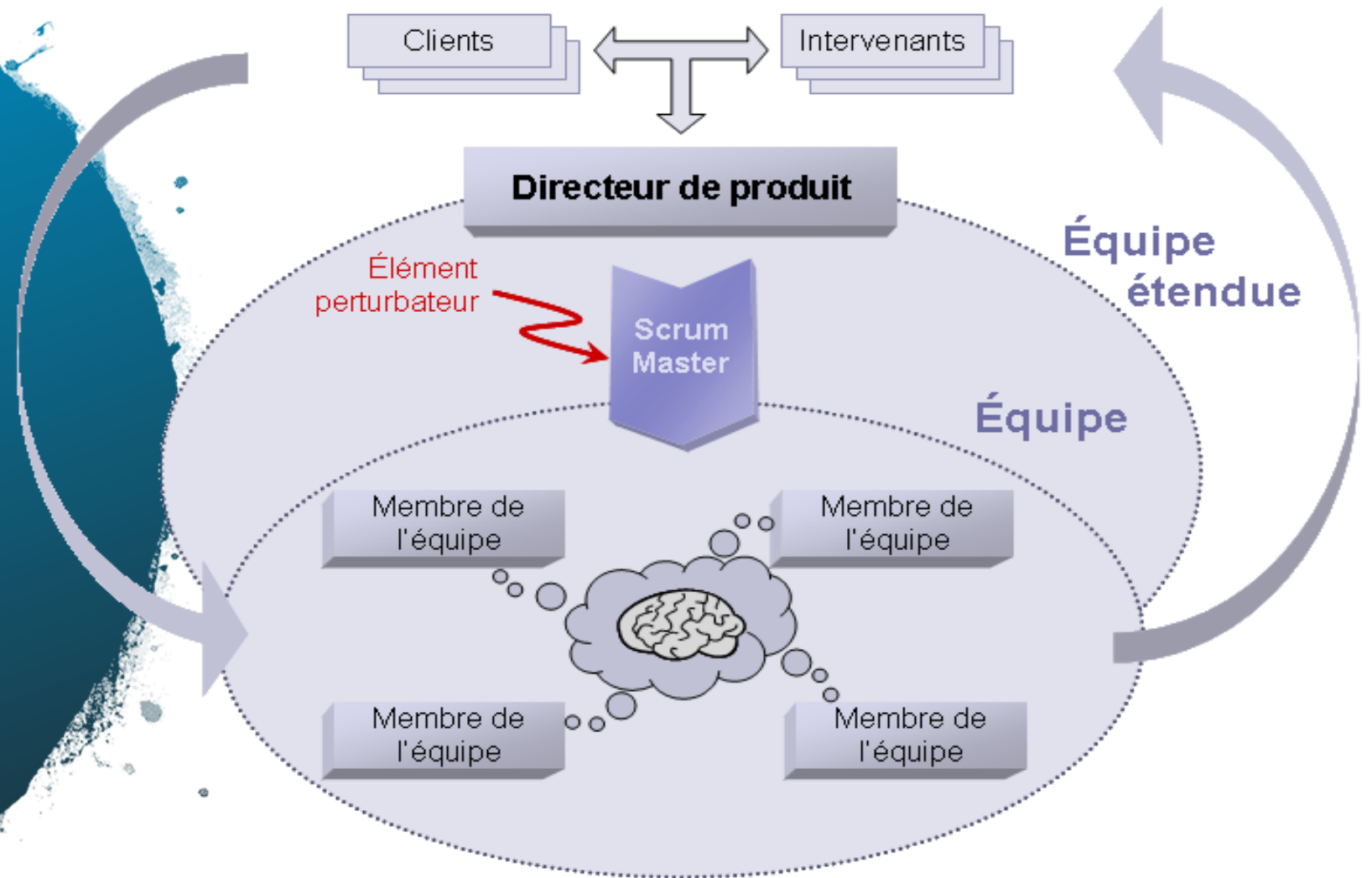
Cérémonial

- Planification du sprint
- Revue du sprint
- Rétrospective
- Scrum quotidien

Artefacts

- Backlog de produit
- Liste des tâches
- Burndowns

Rôles



Rôles

Product Owner

- Le directeur de produit ou Product Owner représente les clients ou les utilisateurs finaux.
- Définit les fonctionnalités du produit
- Choisit la date et le contenu de la release
- Définit les priorités dans le backlog en fonction de la valeur « métier »
- Ajuste les fonctionnalités et les priorités à chaque sprint si nécessaire
- Accepte ou rejette les résultats

Qualités nécessaires

- ✓ Bonne communication.
- ✓ Savoir exprimer ses besoins sans ambiguïté.



Rôles

ScrumMaster



- ScrumMaster est l'animateur de l'équipe faisant appliquer Scrum Élimine les obstacles
- S'assure que l'équipe est complètement fonctionnelle et productive
- Facilite une coopération poussée entre tous les rôles et fonctions
- Protège l'équipe des interférences extérieures

Qualités nécessaires

- Bonne communication.
- Savoir résoudre des conflits et des problèmes afin de ne pas gêner l'équipe par rapport aux éléments extérieurs.
- Savoir mener sans imposer ses idées puisque chaque décision relève de l'équipe complète.

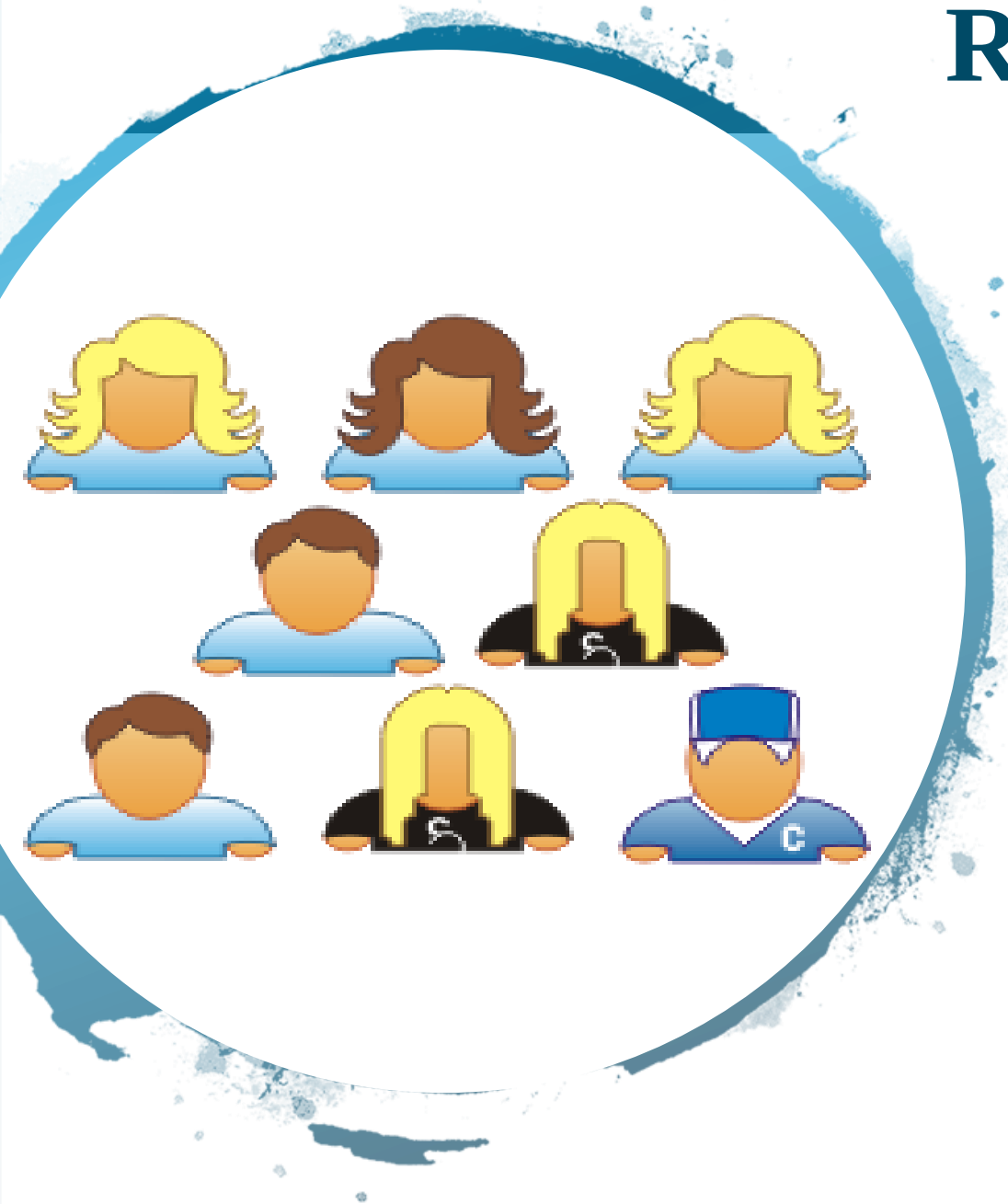
Rôles

Equipe Scrum

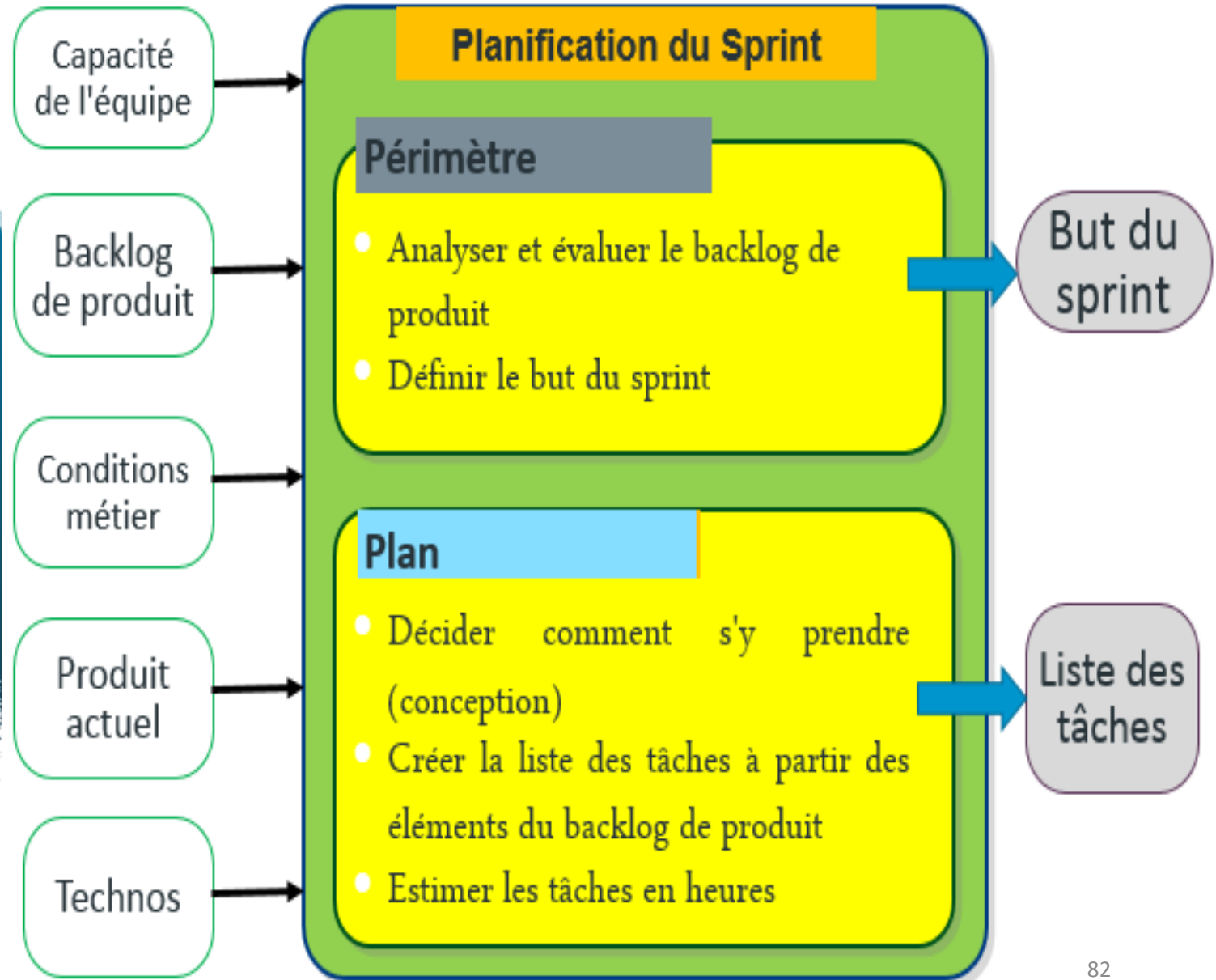
- De 5 à 10 personnes
- Regroupant tous les rôles
- L'équipe s'organise par elle-même
- La composition de l'équipe ne doit pas changer pendant un Sprint

Qualités nécessaires

- ✓ Savoir communiquer avec le reste de l'équipe et le directeur de produit.
- ✓ Ne pas s'imposer au sein de l'équipe. Tout le monde est au même niveau.



Les cérémonials





Les cérémonials

Planification du sprint

- L'équipe choisit, à partir du backlog de produit, les éléments qu'elle s'engage à finir.
- La liste des tâches est créée
 - Les tâches sont identifiées et estimées
 - Collectivement, pas seulement par le ScrumMaster
- La conception de haut niveau est abordée

Les cérémonials

Scrum quotidien (stand-up meeting)

- Paramètres
 - ✓ Tous les jours
 - ✓ 15 minutes
 - ✓ Debout
- Pas fait pour résoudre les problèmes
 - ✓ Tout le monde est invité
 - ✓ Seuls les membres de l'équipe peuvent parler
- Permet de se coordonner, de s'exprimer (rencontre de problèmes), de connaître l'avancement au quotidien



Les cérémonials

Chacun répond à 3 questions

Qu'as-tu fait hier ?

Que vas-tu faire aujourd'hui ?

Y a t-il un obstacle qui te freine ?

Les cérémonials

Revue de sprint



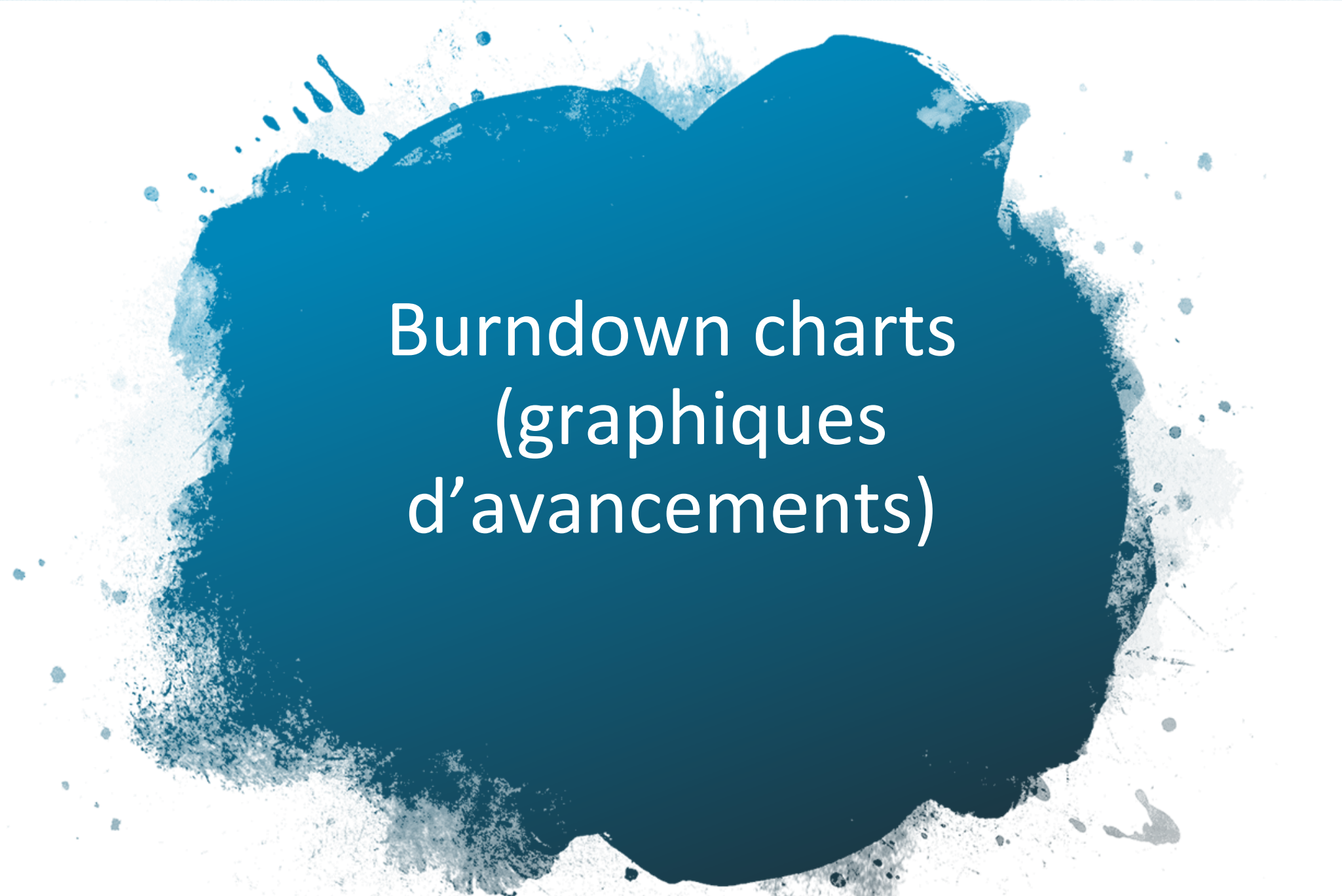
- L'équipe présente ce qu'elle a fait pendant le sprint
- Se fait avec une démo des nouvelles fonctionnalités ou de l'architecture
- Informel
 - Préparation < 2h
 - Pas de slides
- Toute l'équipe participe
- On invite du monde



Les cérémonials

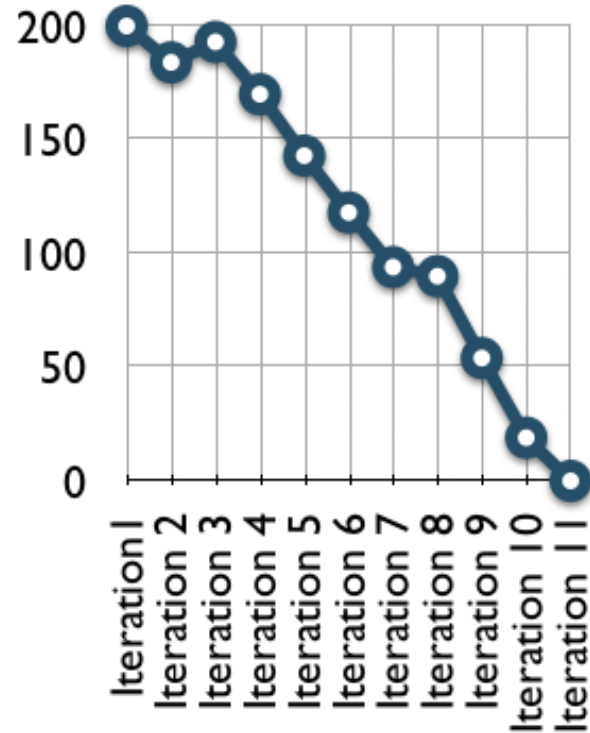
Rétrospective du sprint

- Réfléchir régulièrement à ce qui marche et ce qui ne marche pas
- Dure en général de 15 à 30 minutes
- Fait à la fin de chaque sprint
- Toute l'équipe participe
- ScrumMaster
- Product Owner
- Equipe
- Eventuellement clients et autres intervenants



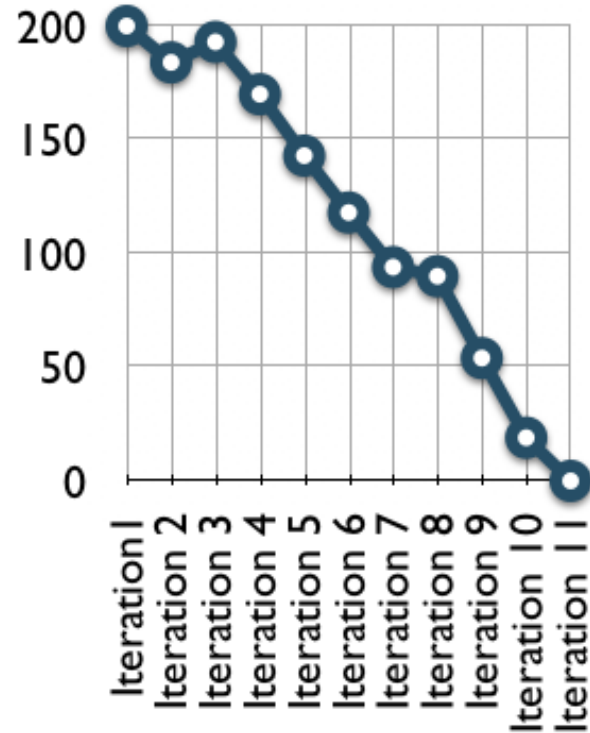
Burndown charts (graphiques d'avancements)

Release Burndown Chart



- Les Burndowns montrent ce qu'il reste à faire en terme d'estimation en nombre d'histoire à réaliser sur la release.
- Ce burndown montre qu'il y a 200 histoires (fonctionnalités) à réaliser. Au fur et à mesure du temps, on voit le nombre baisser mais il peut augmenter si de nouvelles fonctionnalités ont été ajoutées.

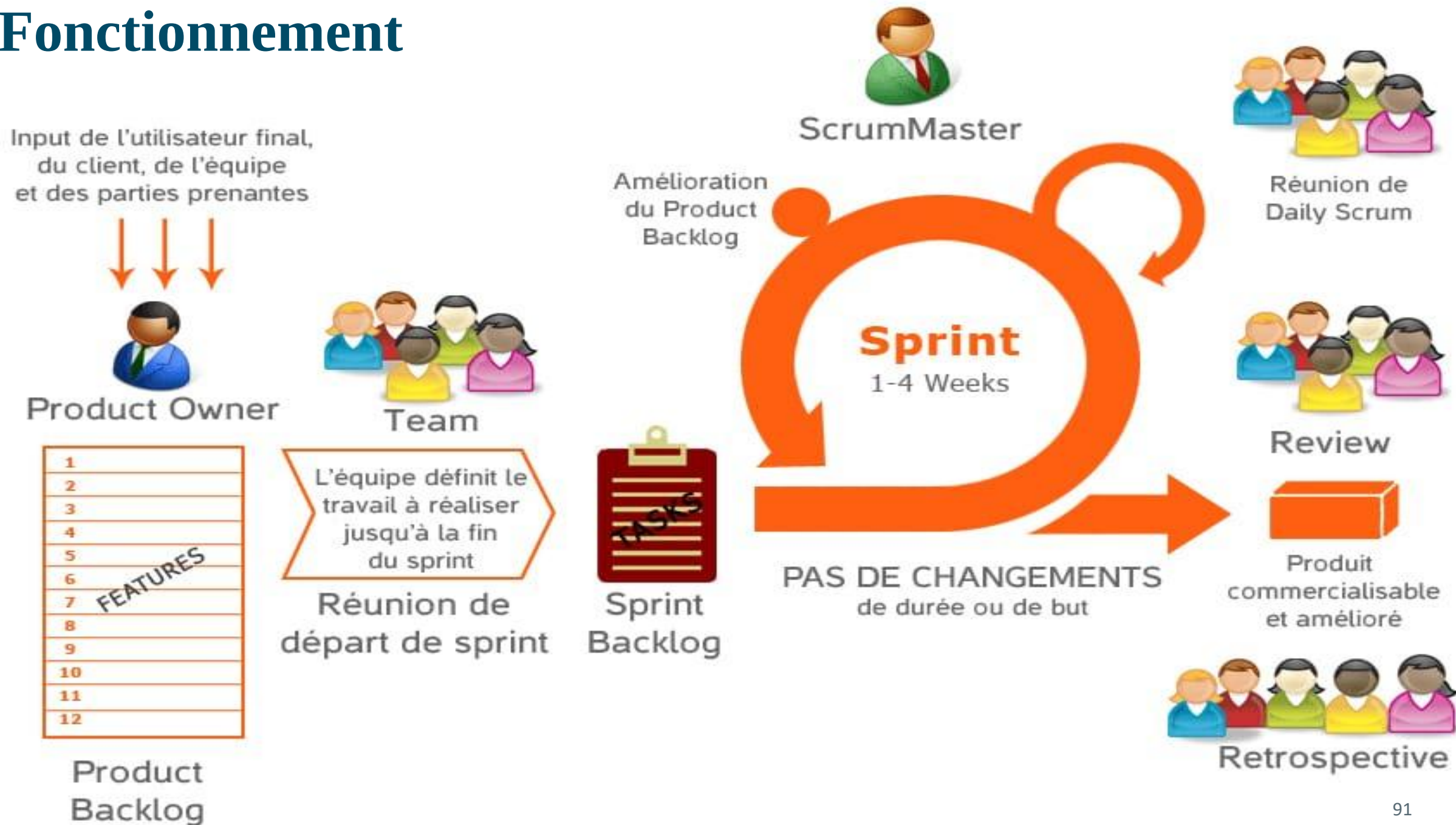
Burndown de release



La quantité totale de points restant à faire dans la release au fil des sprints.

➡ Vue sur l'avancement de la release.

Fonctionnement



Scrum

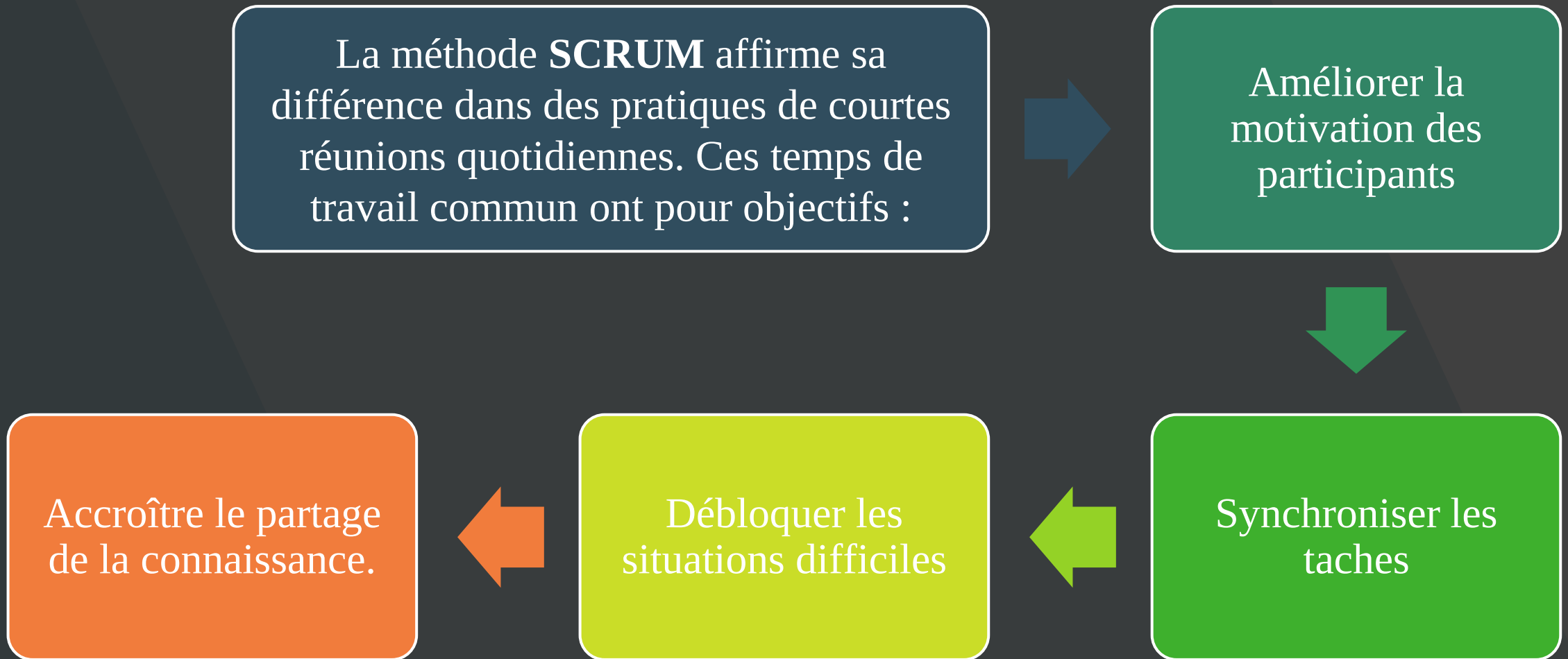
Avantages

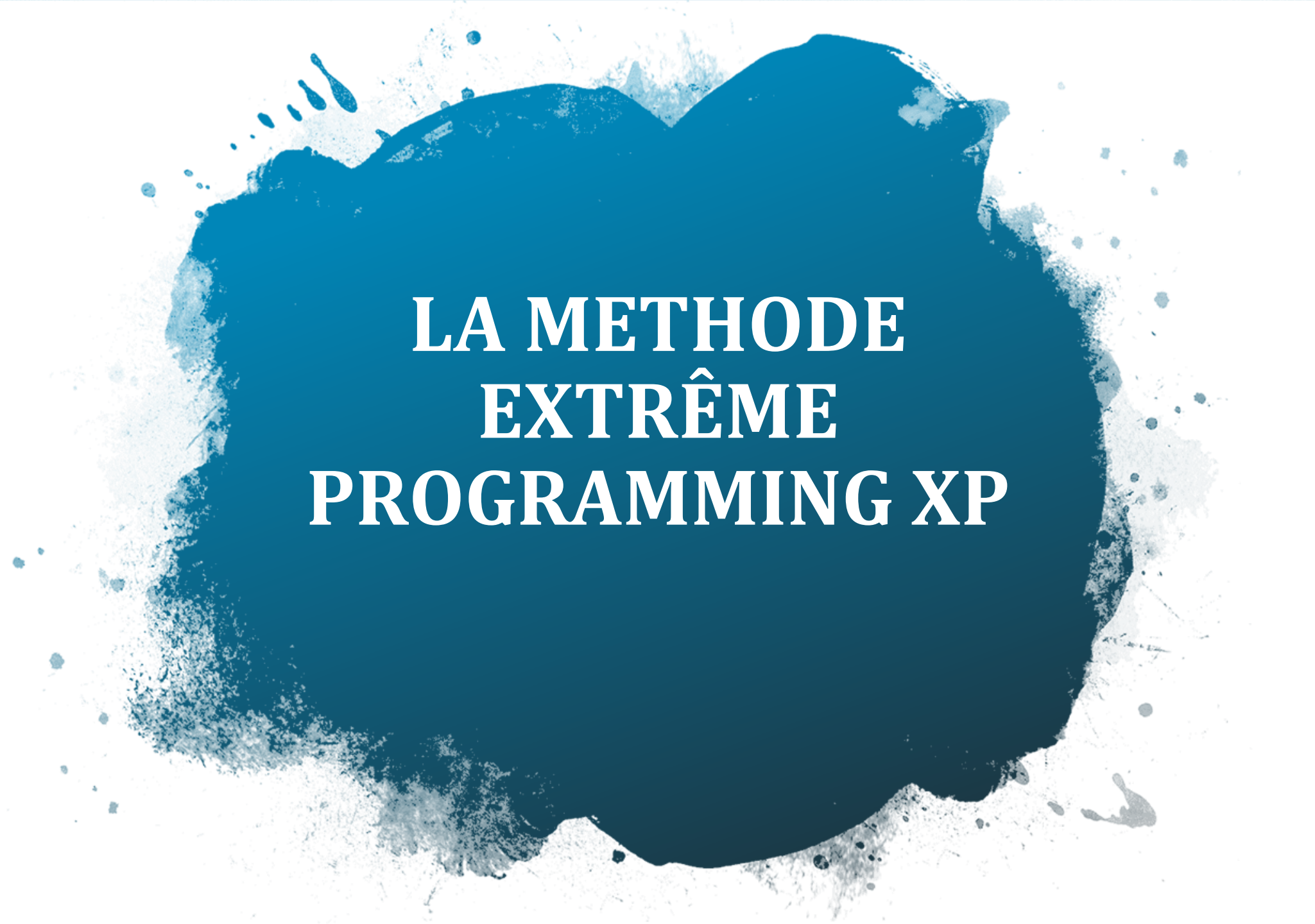
- Méthode très simple et très efficace
- Adoptée par les géants du marché : Microsoft, Google, Nokia..
- Orientée projet contrairement à XP qui est orientée développement
- Peut inclure d'autres activités venant d'autres méthodologies (surtout XP)

Inconvénients

- N'est pas 100% spécifique au GL
- Difficulté de budgétiser un projet

Conclusion





LA METHODE EXTRÊME PROGRAMMING XP

Définitions

XP (Extreme Programming)

- est une méthodologie agile plus radicale, se concentrant plus sur le processus d'ingénierie logicielle et en abordant les phases d'analyse, de développement et de test avec des approches novatrices qui font une différence substantielle à la qualité du produit final.
- est une approche pragmatique du développement de programmes qui met l'accent sur les résultats d'affaires d'abord et prend une approche incrémentielle, get-something-started à la construction du produit, en utilisant des tests continus et de révision.

Origine

Le développement de logiciels dans les années 1990 a été influencé par deux influences majeures :

- A l'interne, la programmation orientée objet a remplacé la programmation procédurale comme le paradigme de programmation privilégié par certains dans l'industrie ;
- À l'extérieur, la montée de l'Internet et le boom des points-com ont mis l'accent sur la rapidité de mise sur le marché et la croissance de l'entreprise en tant que facteurs commerciaux concurrentiels.

L'évolution rapide des exigences exigeait des cycles de vie des produits plus courts et était souvent incompatible avec les méthodes traditionnelles de développement de logiciels.

Origine

- Le système Chrysler Comprehensive Compensation (C3) a été lancé afin de déterminer la meilleure façon d'utiliser les **technologies d'objet**, en utilisant les systèmes de paie de Chrysler comme l'objet de la recherche, avec **Smalltalk** comme langue et **GemStone** comme couche d'accès aux données.
- Ils ont amené **Kent Beck**, un pratiquant de Smalltalk, pour faire l'ajustement des performances sur le système, mais son rôle a augmenté comme il a noté plusieurs problèmes qu'ils ont eu avec leur processus de développement.
- Il a profité de cette occasion pour proposer et mettre en œuvre certains changements dans ses pratiques basées sur son travail avec son collaborateur fréquent, **Ward Cunningham**. Beck décrit la conception précoce des méthodes :

Origine

- La première fois qu'on m'a demandé de diriger une équipe, j'ai leur demandé de faire un peu des choses, j'ai pensé qu'ils étaient raisonnables, comme des tests et des examens.
- La deuxième fois il y avait beaucoup plus sur la ligne. J'ai demandé à l'équipe de mettre en marche tous sur les choses que je croyais essentielles et laisser de côté tout le reste.
- Beck a invité **Ron Jeffries** au projet pour aider à développer et affiner ces méthodes. Jeffries a ensuite agi comme un coach pour instiller les pratiques comme habitudes dans l'équipe C3.



Origine

La méthode Extreme Programming a été inventée par Kent Beck, Ward Cunningham et Ron Jeffries

But de la méthode

La méthode Extreme Programming est décrit comme étant :

- Une tentative de concilier humanité et productivité
- Un mécanisme de changement social
- Un chemin vers l'amélioration
- Un style de développement
- Une discipline de développement logiciel

L'objectif principal de XP est de réduire le coût du changement.

Valeurs de la méthode XP

Communication

Feedback

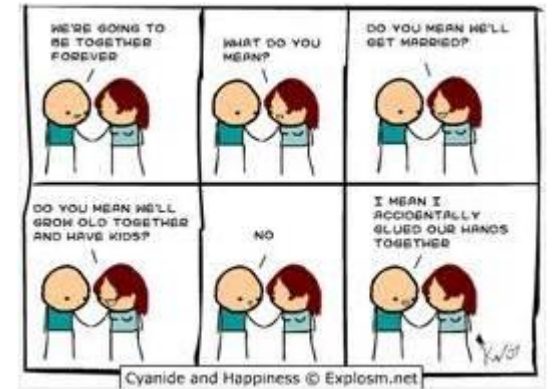
Simplicité

Courage

Respect

Valeurs de la méthode XP

Communication



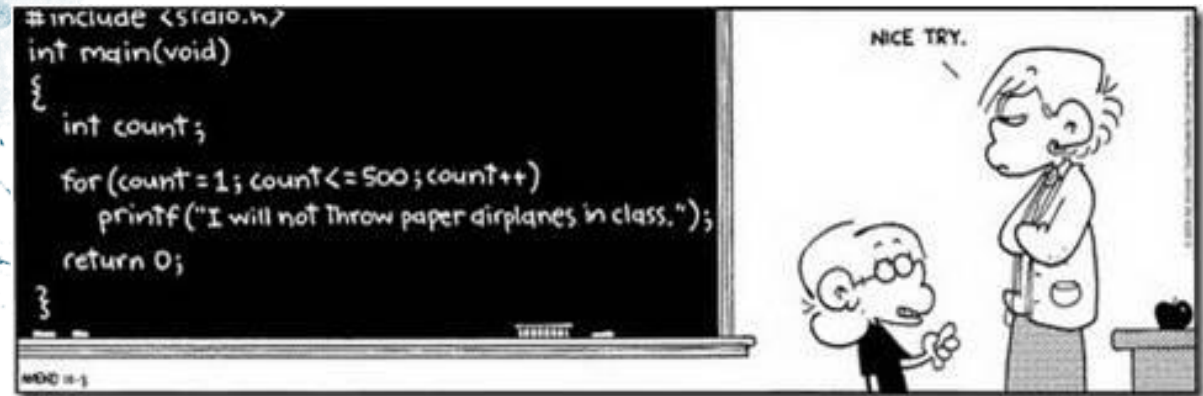
La communication dans XP est bidirectionnelle et repose sur un système de petites boucles de Retour d'information. Dans un projet XP, les règles de communication sont simples :

- Tous les canaux sont ouverts en tout temps
- Le client est libre de parler aux programmeurs.
- Les programmeurs parlent au client et aux utilisateurs.
- Une communication sans entrave atténue le risque de projet en réduisant les fausses attentes. Tous les intervenants savent ce qu'ils peuvent attendre du reste de l'équipe.

Valeurs de la méthode XP

Simplicité

- XP favorise une approche détonante du code source (c'est-à-dire écrire, lancer, réécrire).
- Le plus simple fait partie de la solution ; Mais aussi le travail est le code qui a passé les tests est la première règle de la conception simple. Cela signifie également que chaque exigence, fonctionnelle ou non fonctionnelle, doit être exprimée comme un test.



Sans tests, personne ne sait si la complexité ajoutée vaut la peine.

Valeurs de la méthode XP

Retour d'information

Les retours d'information portent sur différentes dimensions du développement du système :

➤ Retour d'information du système

- En écrivant des tests unitaires, ou en exécutant des tests d'intégration périodiques, les programmeurs ont une rétroaction directe de l'état du système après la mise en œuvre des changements.

➤ Retour d'information du client

- Les tests fonctionnels (tests d'acceptation) sont rédigés par le client et les testeurs. Ils obtiendront des retours d'information concrets sur l'état actuel de leur système. Cette revue est prévue une fois toutes les deux ou trois semaines afin que le client puisse facilement diriger le développement.

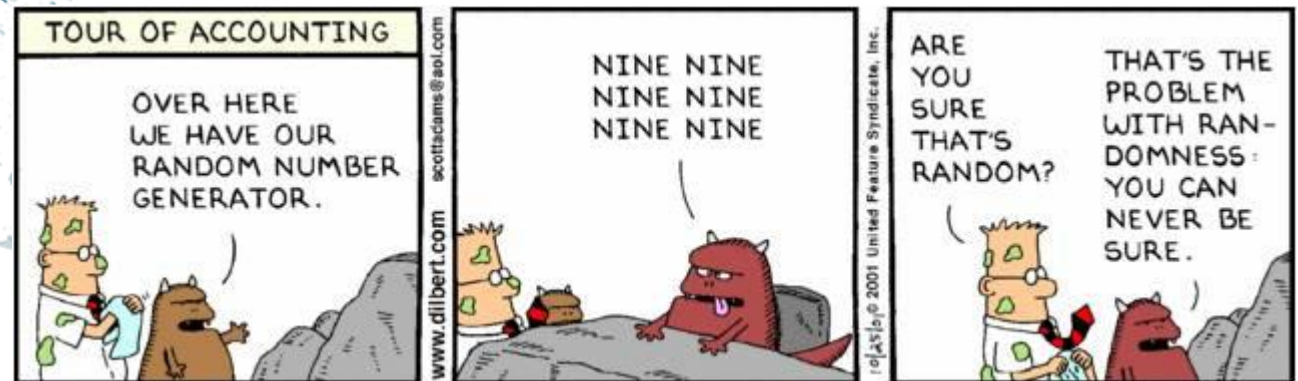
➤ Retour d'information de l'équipe

- Lorsque les clients présentent de nouvelles exigences dans le jeu de planification, l'équipe donne directement une estimation du temps qu'il faudra mettre en œuvre.

Valeurs de la méthode XP

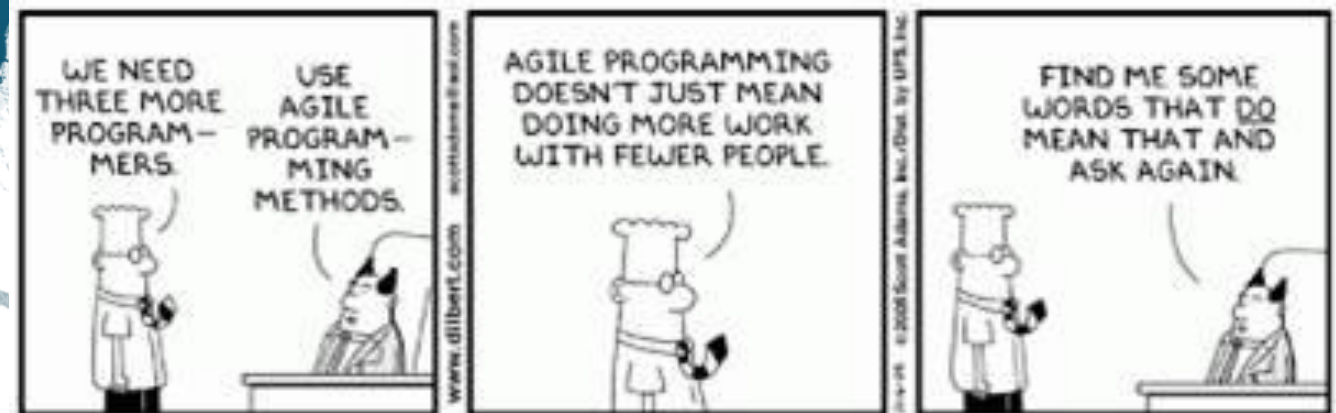
Courage

- L'un est le commandement de toujours concevoir et coder pour aujourd'hui et non pour demain. C'est un effort pour éviter de s'embourber dans la conception et nécessitant beaucoup d'efforts pour mettre en œuvre autre chose.
- Courage permet aux développeurs de se sentir à l'aise avec le refactorisation de leur code si nécessaire. Cela implique de revoir le système existant et de le modifier afin que les modifications futures puissent être mises en œuvre plus facilement.



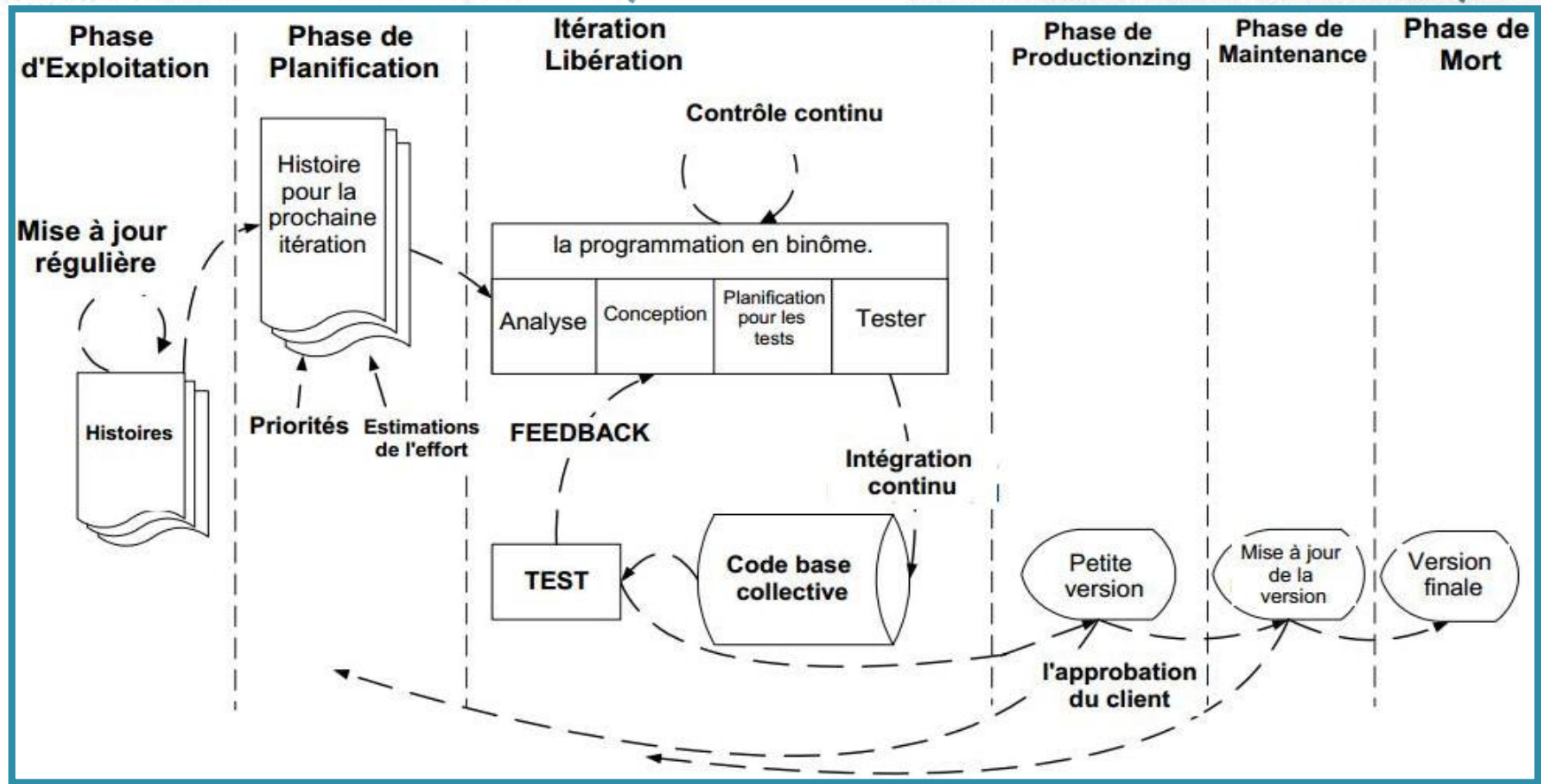
Valeurs de la méthode XP

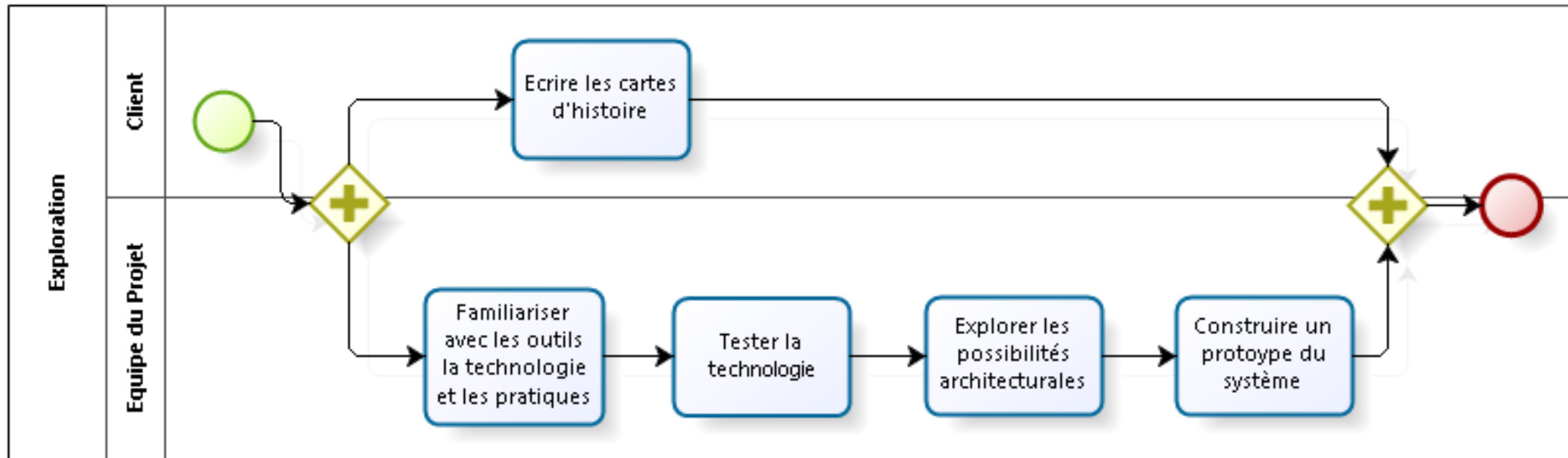
- Respect
- Dans Extreme Programming, les membres de l'équipe respectent les uns les autres parce que les programmeurs ne doivent jamais commettre des changements qui cassent la compilation, qui font échouer les tests unitaires existants ou qui retardent autrement le travail de leurs pairs.
- Les membres respectent leur travail en s'efforçant toujours de la qualité et en recherchant la meilleure conception pour la solution à la main en refactorisation.



Cycle de vie de la méthode XP

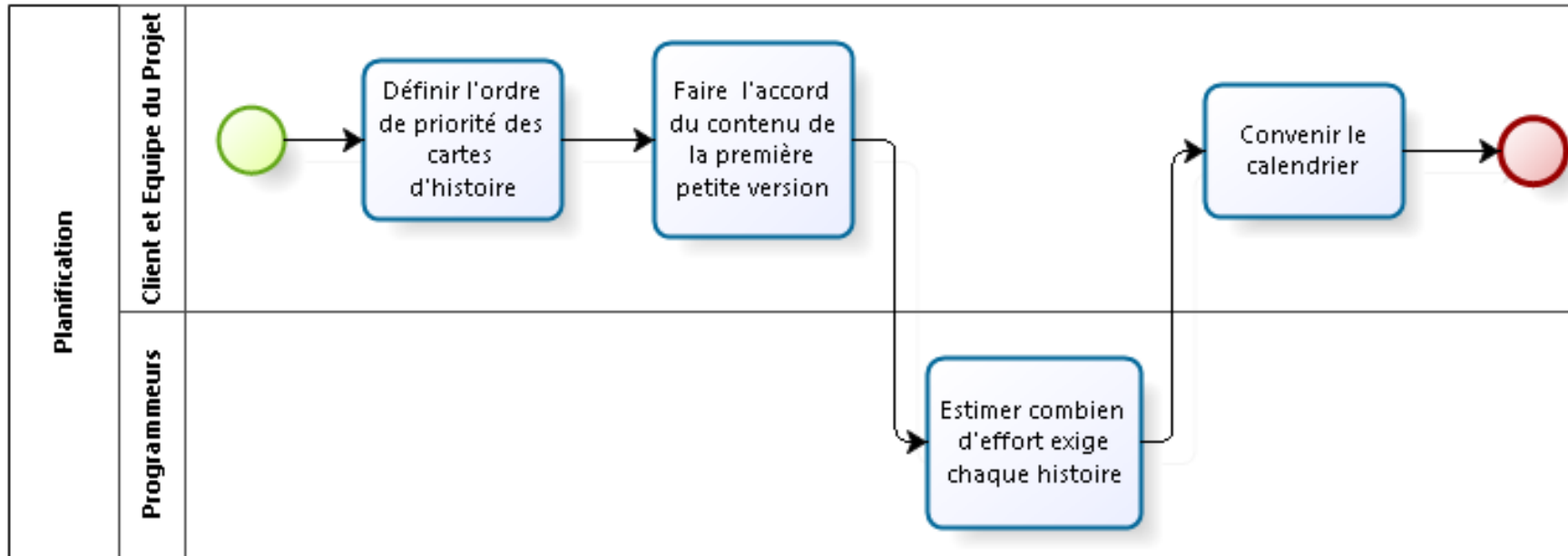
Cycle de vie





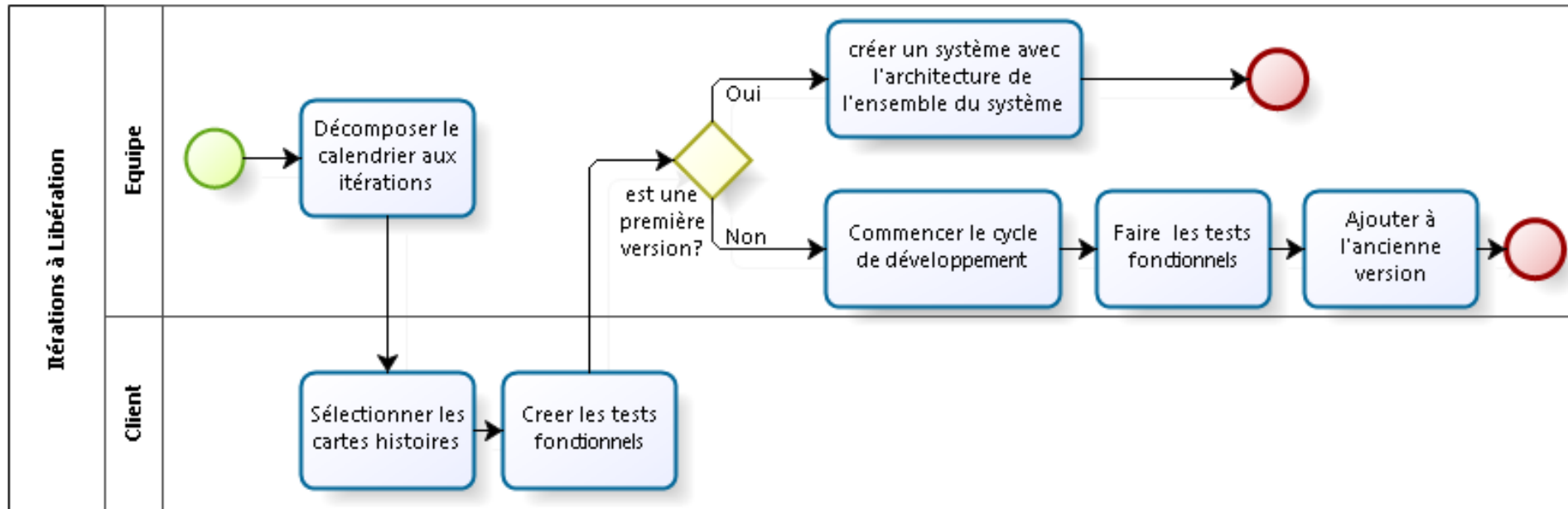
Phase d'Exploration

- La phase d'exploration prend entre quelques semaines à quelques mois, dépendant en grande partie sur la façon dont la technologie familière est aux programmeurs.



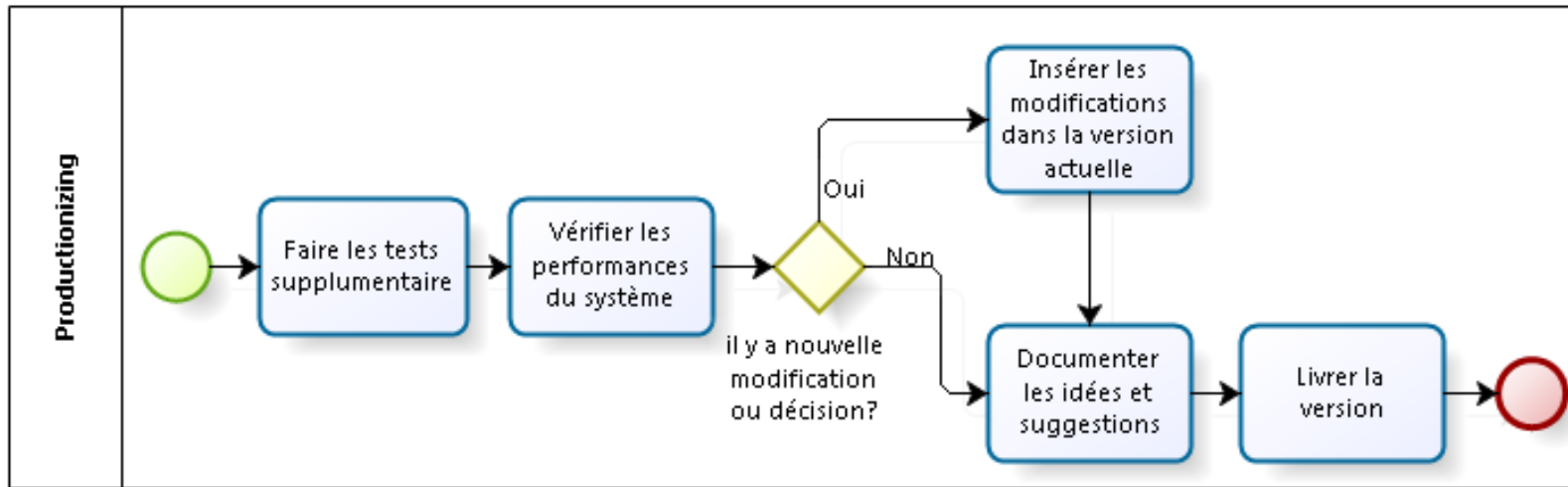
Phase de Planification

La période de temps de l'horaire de la première version ne dépasse normalement pas deux mois. La phase de planification lui-même prend quelques jours.



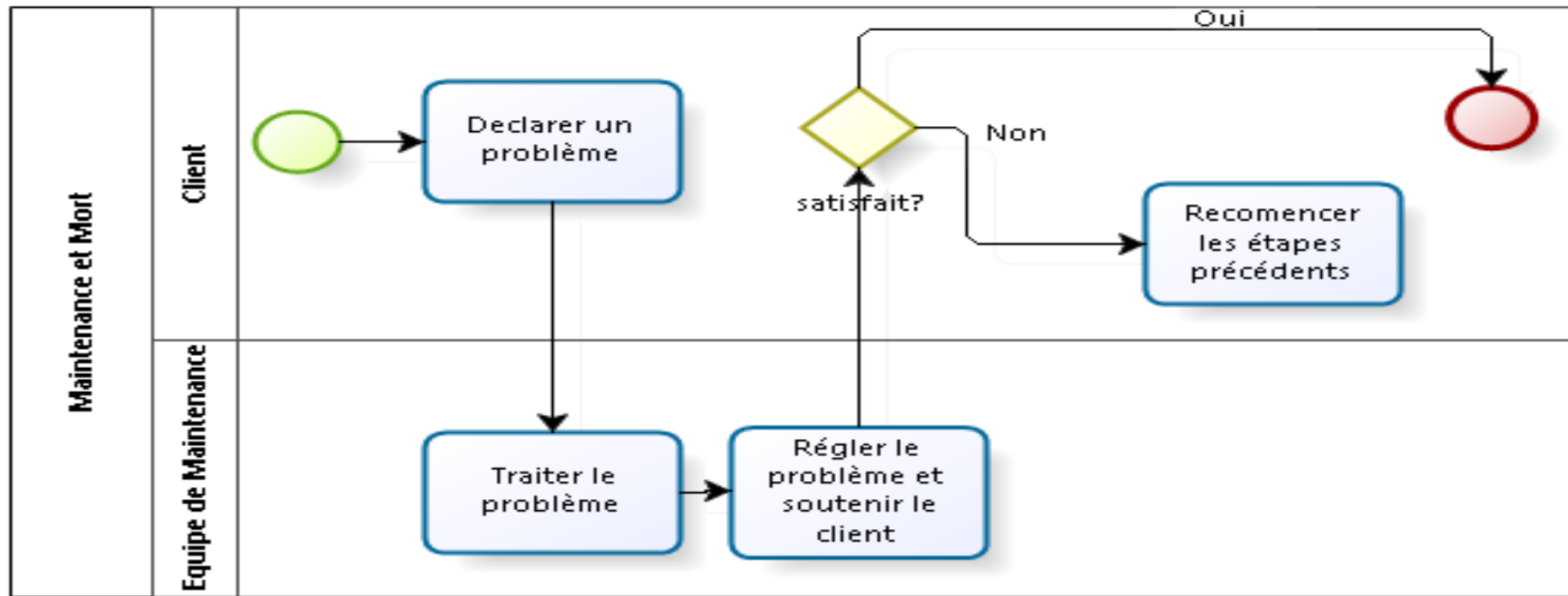
Phase d'Itérations à libération

- Les tests fonctionnels créés par le client sont exécutés à la fin de chaque itération. A la fin de la dernière itération, le système est prêt pour la production.



Phase Mise en production

- Au cours de cette phase, les itérations peuvent être besoin d'être accélérer de trois semaines à une semaine. Les idées et suggestions reportées sont documentées pour la mise en œuvre plus tard au cours, par exemple, la phase d'entretien.



Phase de maintenance et de mort

- Après la première version est produite pour une utilisation du client, le projet de XP doit à la fois maintenir le système dans la production en cours d'exécution tout en produisant de nouvelles itérations.

Rôles et responsabilités

Client

- Le client définit ce qu'il faut faire (**User Stories**) et dans quel ordre (**Jeux de planification**).
- XP a une règle spéciale appelée **client sur site**.

client sur site : un représentant du client, généralement un expert du domaine qui est un utilisateur potentiel du système, fait partie de l'équipe de développement, et donc au développement "site".

Programmeur

- Il ne fait pas recours à des spécialistes comme les analystes, architectes logiciels ou concepteurs de logiciels. Au lieu de ce travail est effectué par les programmeurs. Il y a plusieurs compétences que le programmeur doit avoir :

La communication

- XP repose sur la communication, en particulier la communication face-à-face.

Codage

- Le programmeur doit être raisonnablement bon au codage.

Capacité de travailler en équipe

- Ceci est une compétence évidente pour une méthodologie axée sur l'équipe comme XP.
- Mais cela signifie plus, surtout avec **propriété collective du code** et de la **programmation par paires**.

Entraîneur (Coach)

La partie gestion d'un projet XP est divisée en deux rôles : l'entraîneur et le traqueur.

1) L'entraîneur

- L'entraîneur est responsable de l'exécution technique (et évolution) du projet. Il doit avoir de bonnes compétences en **communication et en technique et confiant**.
- Les fonctions de l'entraîneur sont :
 - Expliquant le processus de gestion et de clients ;
 - Fournir des compétences techniques comme les tests, la mise en forme et refacturation ;
 - Ayant un aperçu général du système - en particulier pour les objectifs de refacturation à long terme ;
 - Disponible en tant que partenaire de développement (programmation en paires) surtout pour les nouveaux membres de l'équipe.
- L'entraîneur participe également aux réunions de gestion du projet.

Entraîneur (Coach)

2) Traqueur (Tracker)

- Le travail du traqueur est de rassembler tout ce que les **métriques** sont suivis pour le projet au moins le facteur de charge et les résultats des tests fonctionnels.
- Il est de la responsabilité du suivi afin de déterminer si l'annexe Itération et calendrier d'engagement peuvent être satisfaits.
- Les données sont également utilisées pour donner une **rétroaction** aux développeurs sur la qualité de leurs estimations.
- En outre, les résultats des tests fonctionnels sont rassemblés avec le testeur.

Testeur (tester)

- Contrairement à d'autres procédés, le testeur ne dispose que peu de compétences.
- Ceci est dû au fait que la plupart des tests unitaires (tests white box) est réalisée par les programmeurs.
- Le testeur permet au client de choisir et d'écrire les tests fonctionnels et de les exécuter (en particulier les essais qui ne peuvent pas être automatisés).
- Habituellement, le rôle du testeur n'est pas rempli par une personne dédiée, mais par l'un des programmeurs ou le traqueur.

Manager

- Le manager est le supérieur hiérarchique des programmeurs. Il est aussi parfois le supérieur hiérarchique du client. C'est le manager qui s'occupe matériellement des programmeurs (il leur fournit un bureau, des machines ...) et immatérielle (Il recrute des programmeurs ou un testeur).
- Le manager demande aussi des comptes. Il demande à l'équipe XP de faire ses preuves. Les résultats doivent être concrets et visibles, c'est une des promesses de la méthode : productivité, visibilité, rapport fréquent et honnête. Il participe aux séances de planification, peut-être pas à chaque itération mais au moins chaque fois que le plan de livraison doit être modifié.
- Pour que la méthode XP réussisse, le manager exige que les engagements soient respectés, que les avantages promis par la méthode se confirment.
- Le manager participe aux réunions techniques ou réunions d'avancement générales du projet. Il transmet au monde extérieur l'état des lieux du développement logiciel, et résume à l'équipe logicielle les informations en provenance de l'extérieur.

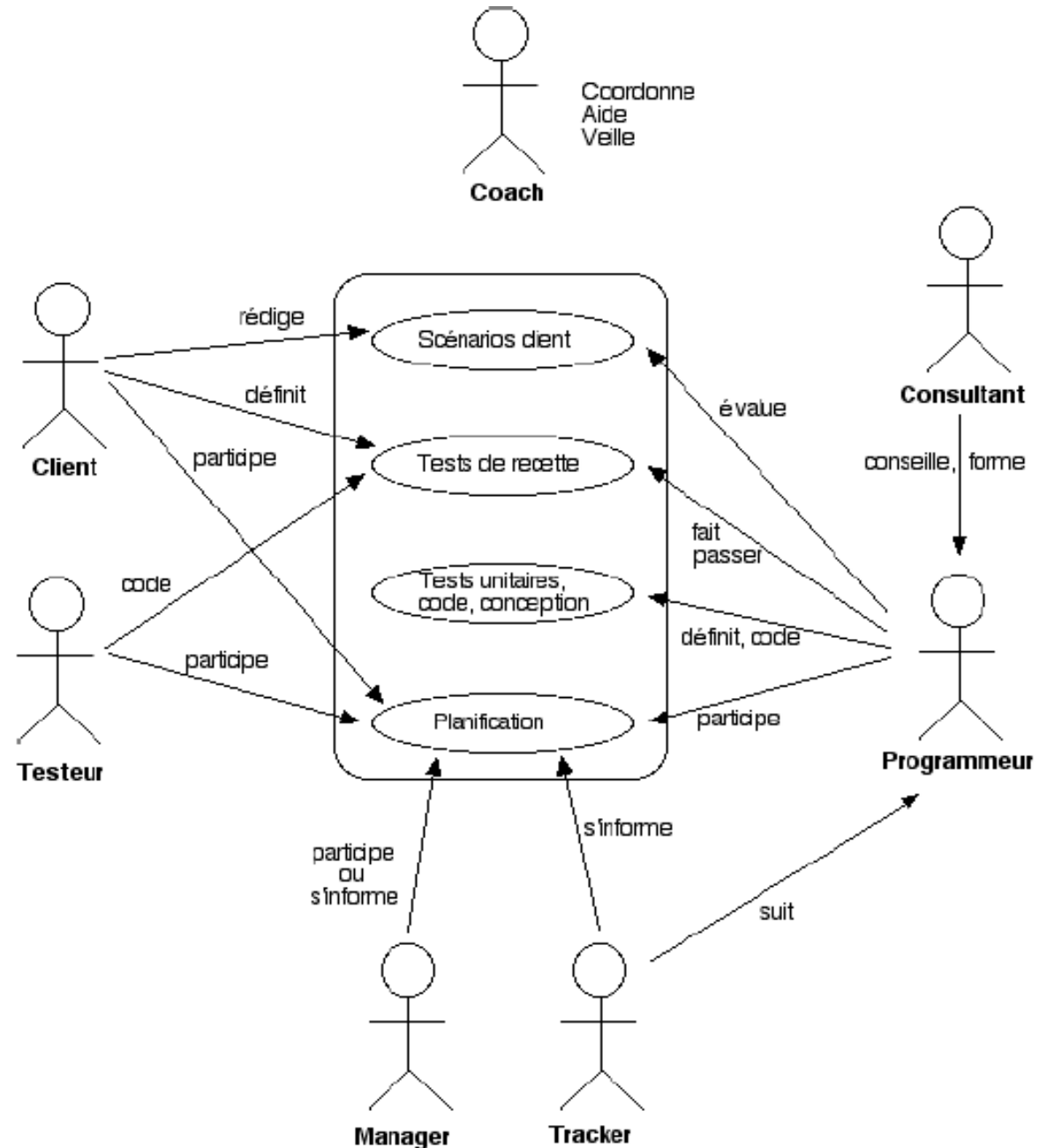
Consultant

En raison des règles et pratiques (**en particulier la programmation en paires**), il n'y a généralement pas de spécialistes dans des projets XP. Mais parfois, le projet a besoin d'une connaissance plus approfondie (technique). Dans ces cas, un consultant sera engagé qui fournit cette connaissance. Cependant, ce rôle est différent dans XP. Le consultant est engagé pour fournir des connaissances. Un ou deux membres de l'équipe seront assis avec le consultant, en posant des questions sur le domaine technique et résoudre le problème. Mais la solution sera refaite après, afin de partager les connaissances entre l'équipe.

Rôles et responsabilités

S

Responsabilités respectives des
rôles XP



Pratiques

Les pratiques de programmation

- **Conception simple (simple design) :** les développeurs implémentent toujours la solution la plus simple qui puisse fonctionner.
- **Refacturation (refactoring) :** les développeurs n'hésitent pas à revenir sur le code écrit pour le rendre plus « propre », le débarrasser d'éventuelles parties inutilisées, et le préparer à l'ajout de la fonctionnalité suivante.
- **Première-Test programmation, tests unitaires, tests de développeur (test-first programming, unit tests, developer tests) :** les développeurs écrivent des tests automatiques pour le code qu'ils produisent, et ce au moment même d'écrire le code en question. Cela leur permet d'une part de mieux cerner le problème avant d'écrire le code, et d'autre part de constituer progressivement une batterie de tests qui les autorise ensuite à apporter rapidement des changements dans l'application, tout en conservant une certaine sérénité.
- **Tests d'acceptation, tests des clients (acceptance tests, customer tests) :** le client précise très explicitement ses besoins et les objectifs des programmeurs – en participant à la rédaction de tests de recette. Comme les tests unitaires, les tests de recette doivent être automatiques afin de pouvoir vérifier tous les jours la non-régression du produit.

Les pratiques de collaboration

- **Programmation en binôme (pair programming)** : lorsqu'ils écrivent le code de l'application, les développeurs travaillent systématiquement à deux sur la même machine – il s'agit là d'une forme «**extrême**» de relecture de code, dans laquelle les deux développeurs collaborent activement pour résoudre les problèmes qu'ils rencontrent. Les binômes changent fréquemment, ainsi chacun est amené à travailler tôt ou tard avec tous les autres membres de l'équipe.
- **Responsabilité collective du code (collective code ownership)** : tous les développeurs de l'équipe peuvent être amenés à travailler sur toutes les parties de l'application. De plus, ils ont le devoir d'améliorer le code sur lequel ils interviennent.

Les pratiques de collaboration

- **Règles de codage (coding standards)** : les développeurs se plient à des règles de codage définies par l'équipe elle-même, de manière à garantir l'homogénéité de leur code avec le reste de l'application.
- **Métaphore (metaphor)** : les développeurs n'hésitent pas à recourir aux métaphores pour décrire la structure interne du logiciel ou ses enjeux fonctionnels, de façon à décrire le système dans son ensemble par une métaphore unique.
- **Intégration continue (continuous integration)** : les développeurs synchronisent leurs développements aussi souvent que possible. Cela réduit la fréquence et la gravité des problèmes d'intégration, et permet de disposer à tout moment d'une version du logiciel qui intègre tous les développements en cours.

Les pratiques de gestion de projet

- **Livraisons fréquentes (frequent releases)** : l'équipe livre des versions du logiciel à un rythme régulier, aussi élevé que possible – la fréquence précise étant fixée par le client.
- **Planification itérative (planning game)** : la planification du projet est réalisée conjointement par le client et l'équipe de développement, au cours de séances dédiées, organisées régulièrement tout au long du projet.
- **Client sur site, équipe entière (on-site customer, whole team)** : le client est littéralement intégré à l'équipe de développement pour arbitrer les priorités, et définir précisément ses besoins en bénéficiant du feedback immédiat d'une application aux livraisons fréquentes.
- **Rythme durable (sustainable pace)** : l'équipe adopte des horaires en conservant leur énergie pour produire un travail de qualité.

Avantages et inconconvénients

Avantages

- Implication active du client
- Forte réactivité des développeurs
- Responsabilisation et solidarité de l'équipe
- Appel aux meilleures pratiques de développement
- Souplesse extrême

Inconvénients

- Demande une certaine maturité des développeurs
- La programmation par paires n'est toujours pas applicable
- Difficulté de planifier et de budgétiser un projet
- Stress dû aux devoirs de l'intégration continue et des livraisons fréquentes
- La faible documentation peut nuire en cas de départ des développeurs

Scrum vs XP

Criteria	Method	SCRUM	XP
Team size		5 – 9	2 – 10
Project size		All	Small
Iteration length (week)		4	2
Roles and responsibilities		√	√
Process centric		X	X
People centric		√	√
Virtual team support		√	X
High Risk Mitigation		√	X
Medium Risk Mitigation		X	√
Documentation		Basic	Basic
Daily Meeting		√	√
Information sharing (via document)		X	X
Meeting face-to-face		X	X
Metamodel-based		X	X